

Basics of Image Processing

Carsten Rother

11/11/2025



UNIVERSITÄT
HEIDELBERG
ZUKUNFT
SEIT 1386



Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

Goal: Find long edge chains

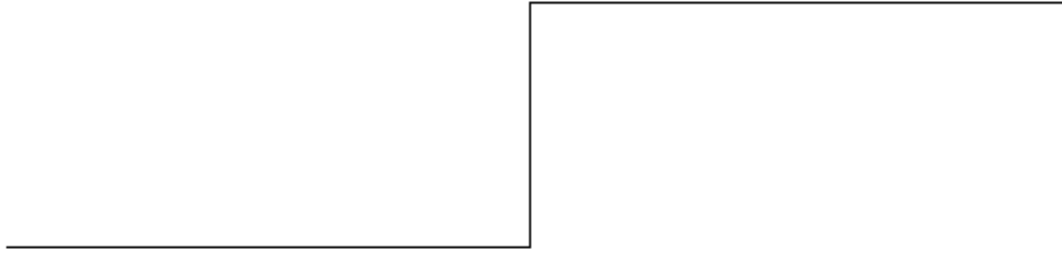
What do we want:

- **Good detection:** we want to find edges not noise
- **Good localization:** find true edge
- **Single response:** one per edge (independent of edge sharpness)
- **Long edge-chains**



Idealized edge types

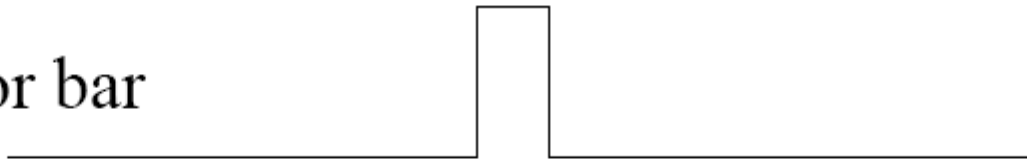
step



ramp



line or bar



roof



We focus
on this

What are edges ?

- correspond to fast intensity changes in the image
- The magnitude of the derivative is large

Image of 2
step edges

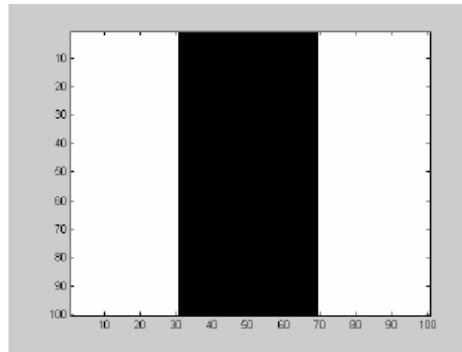
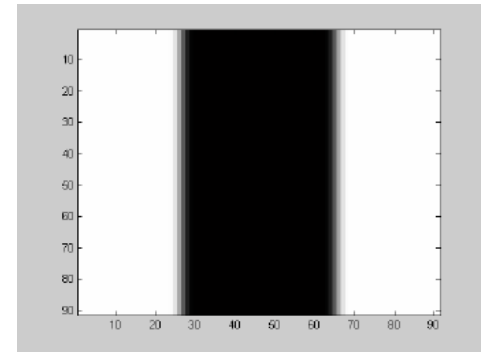
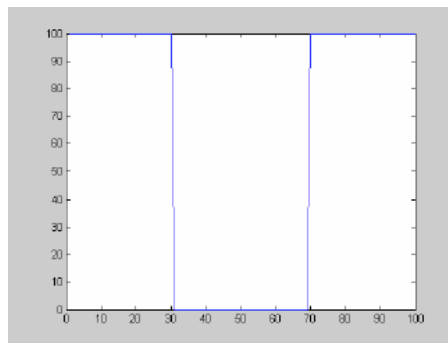


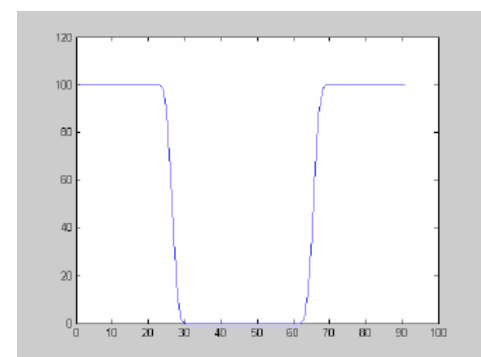
Image of 2
ramp edges



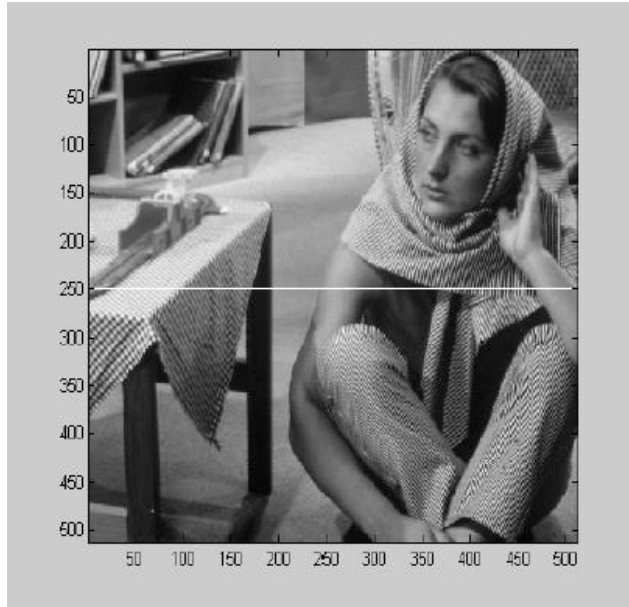
Slice through
the image



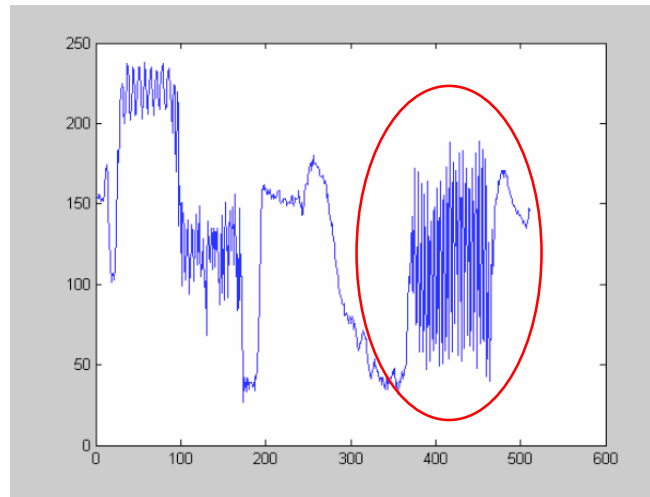
Slice through
the image



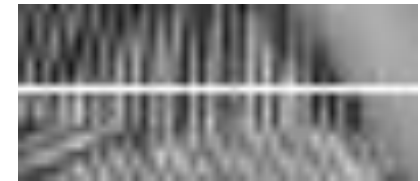
What are fast changes in the image?



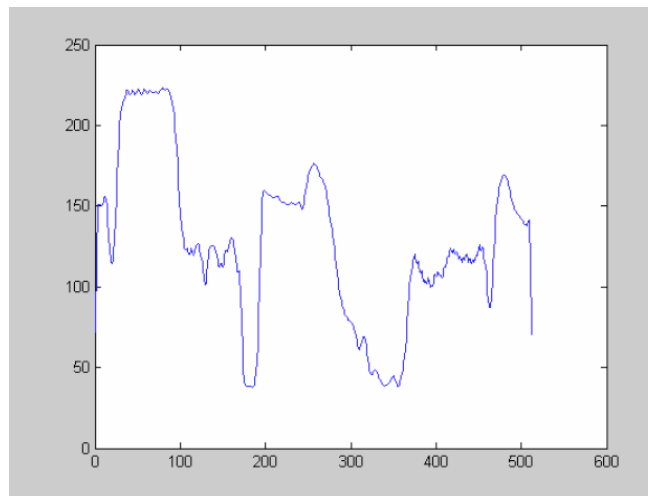
Image



Scanline 250



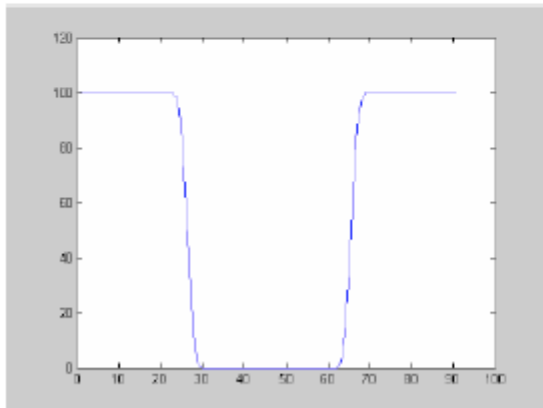
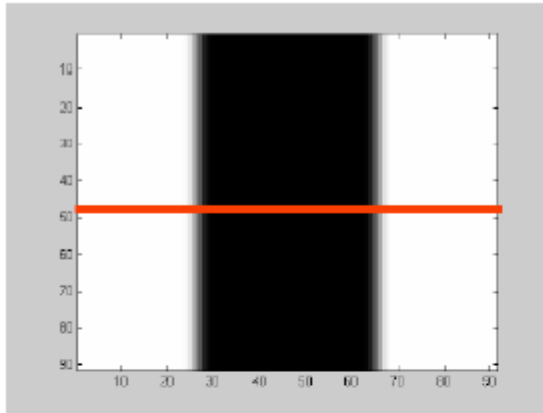
Texture or many edges?



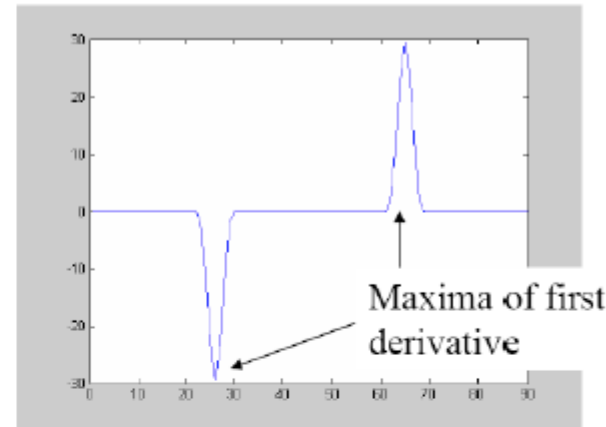
Scanline 250 smoothed with Gaussian

Define edges
after smoothing

Edges and Derivatives



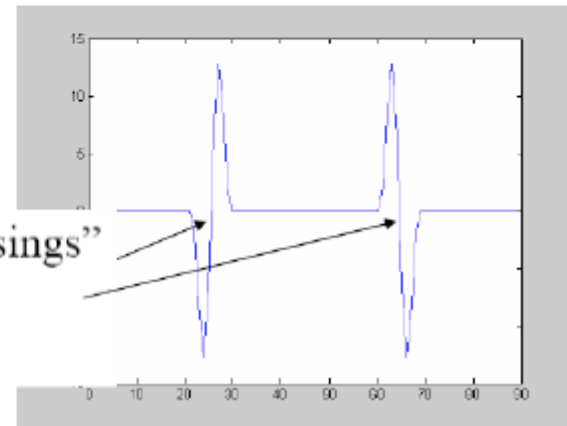
1st derivative



We will
look at
this case

2nd derivative

“zero crossings”
of second
derivative

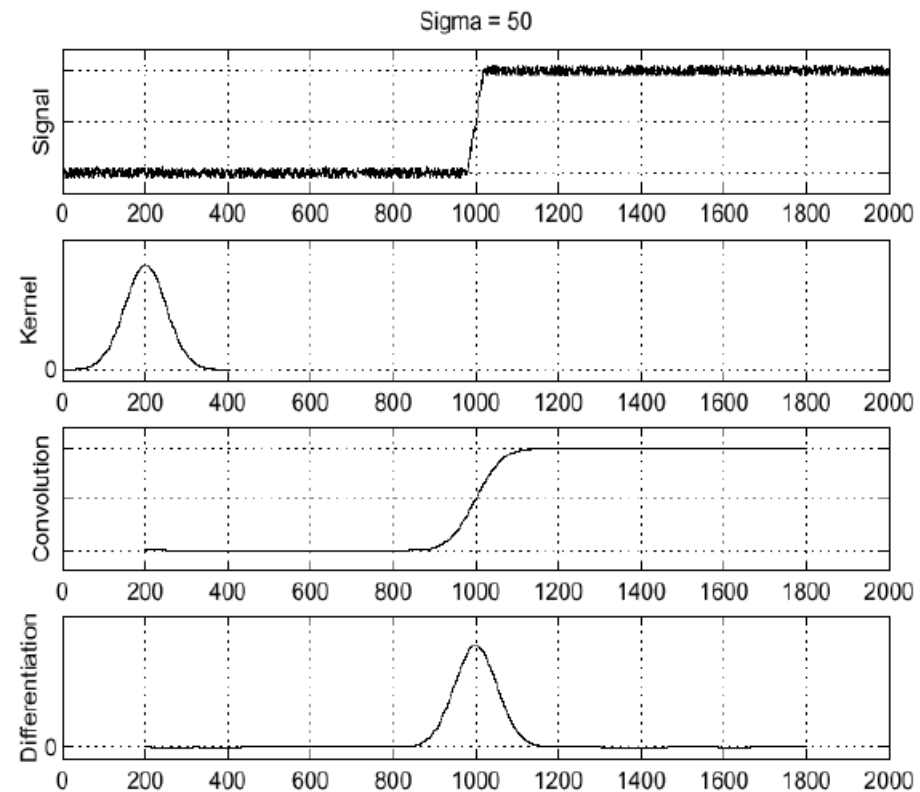


Edge Filter in 1D: Example

Based on 1st derivative

- **Smooth** with Gaussian – to filter out noise
- Calculate **derivative**
- Find its **optima**

$$f$$
$$g$$
$$g * f$$
$$\frac{d}{dx}(g * f)$$



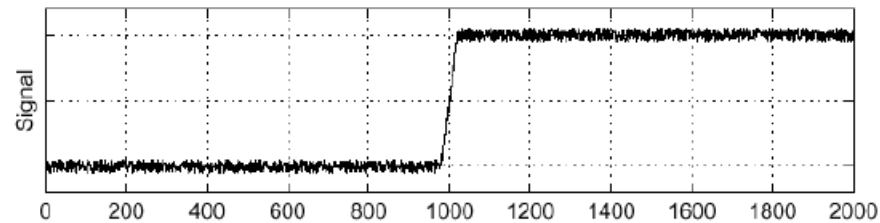
Edge Filtering in 1D

Simplification:
(saves one operation,
since associative)

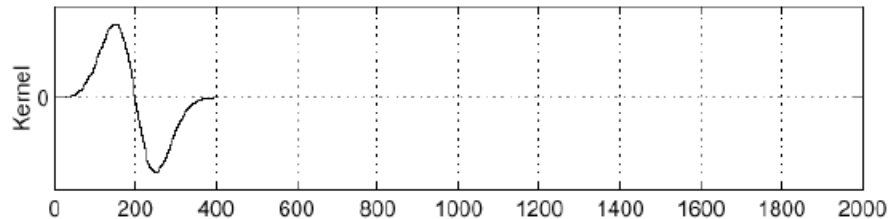
$$\frac{d}{dx} (g * f) = \left(\frac{d}{dx} g \right) * f$$

Derivative of Gaussian

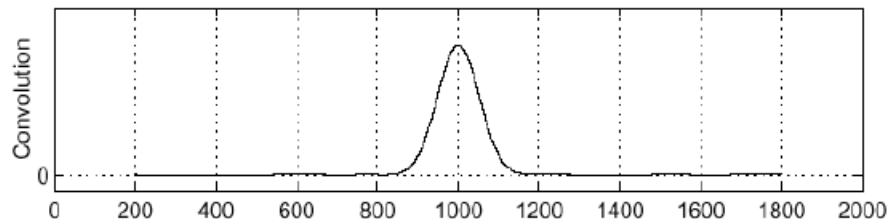
f



$\frac{d}{dx} g$



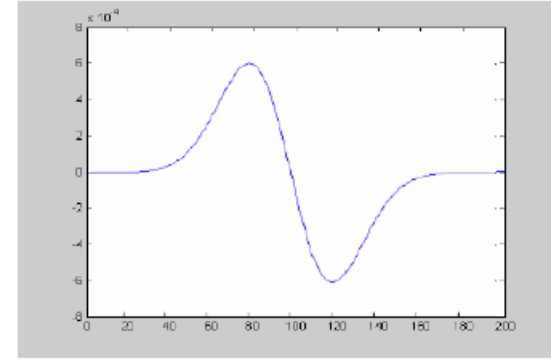
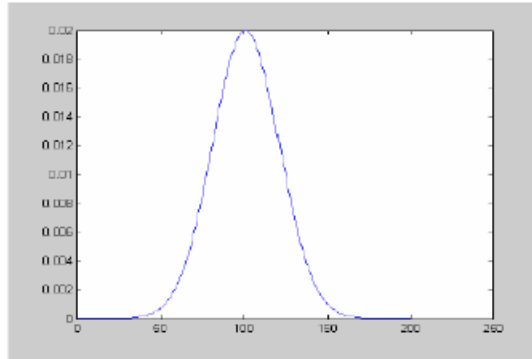
$\left(\frac{d}{dx} g \right) * f$



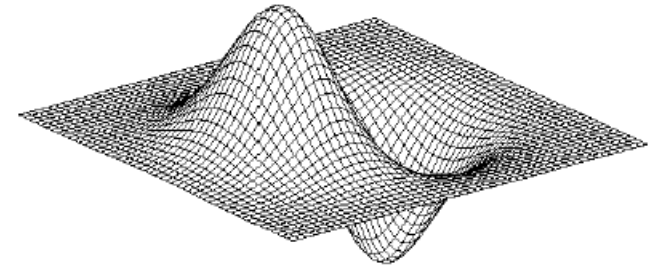
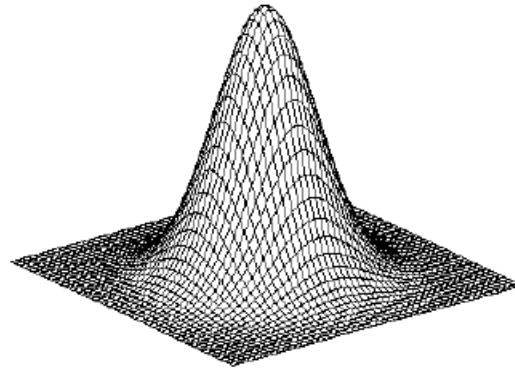
Edge Filtering in 2D

- Derivative in x-direction: $D_x * (G * I) = (D_x * G) * I$

- in 1D:



- in 2D:



Edge filters in 1D

$$\frac{d}{dx}f(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \approx f(x+1) - f(x)$$

We can implement this as a linear filter:

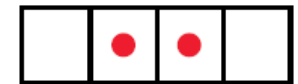
Forward differences: $\frac{1}{2} \begin{bmatrix} -1 & 1 \end{bmatrix}$

Central differences: $\frac{1}{2} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$

Where is the derivative computed?



in between pixels



at pixels

Reminder: Seperable Filters

$$\frac{1}{K^2}$$

1	1	...	1
1	1	...	1
$\vdots$	$\vdots$	1	$\vdots$
1	1	...	1

$$\frac{1}{16}$$

1	2	1
2	4	2
1	2	1

$$\frac{1}{256}$$

1	4	6	4	1
4	16	24	16	4
6	24	36	24	6
4	16	24	16	4
1	4	6	4	1

$$\frac{1}{8}$$

-1	0	1
-2	0	2
-1	0	1

$$\frac{1}{4}$$

1	-2	1
-2	4	-2
1	-2	1

$$\frac{1}{K}$$

1	1	...	1
---	---	-----	---

$$\frac{1}{4}$$

1	2	1
---	---	---

$$\frac{1}{16}$$

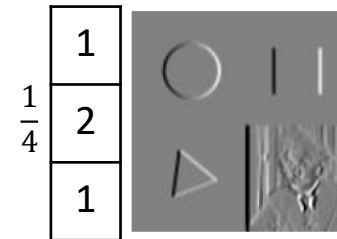
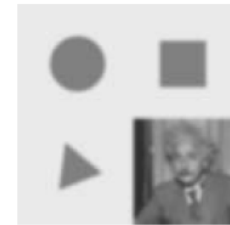
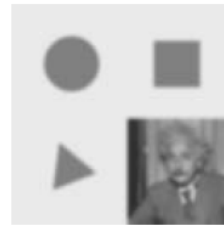
1	4	6	4	1
---	---	---	---	---

$$\frac{1}{2}$$

-1	0	1
----	---	---

$$\frac{1}{2}$$

1	-2	1
---	----	---



(a) box, $K = 5$

(b) bilinear

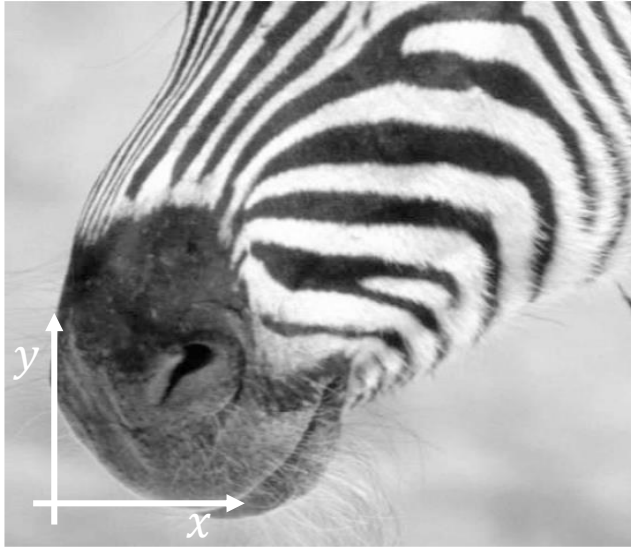
(c) "Gaussian"

(d) Sobel

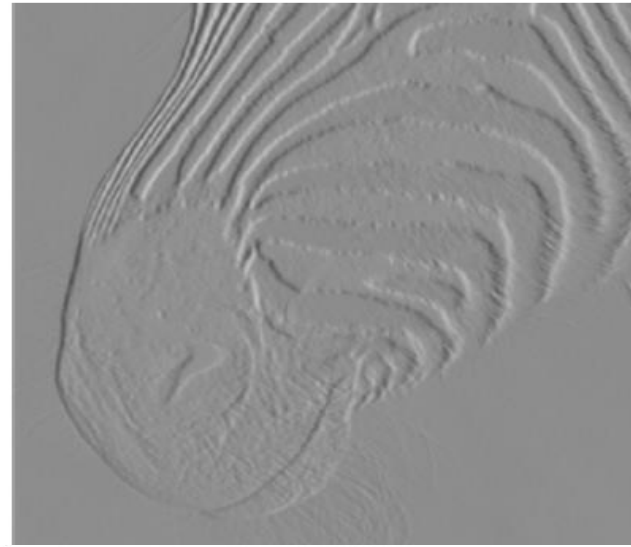
(e) corner

This is the „centralized difference“ operator

Question



Image



First Derivative

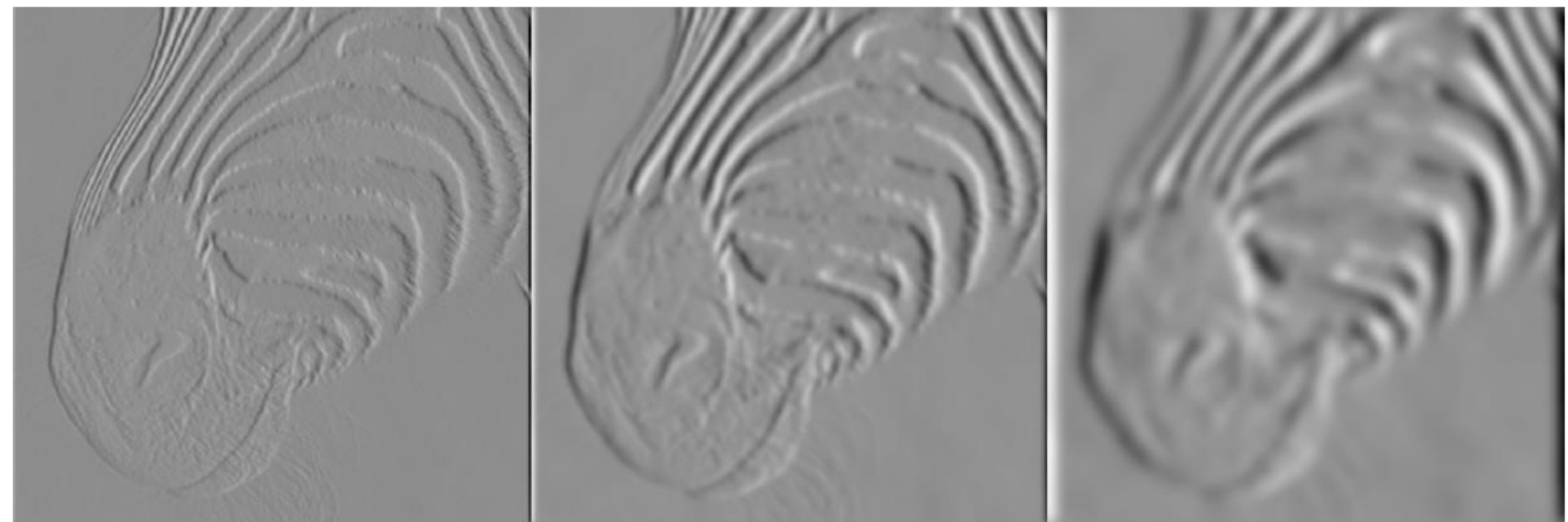
Is the First Derivate Image:

- 1) $D_x * (G * I) ?$
- 2) $D_y * (G * I) ?$
- 3) $(D_x * G) * I ?$
- 4) 1) and 3) ?
- 5) I don't know

Edge Filter in 2D

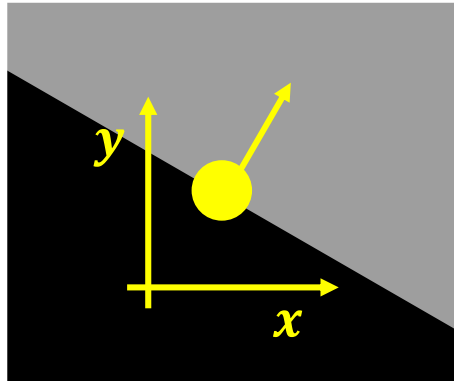


x -derivatives with different Gaussian smoothing



What is a 2D Gradient

$$\left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right) = (k_x, k_y)$$



- Gradient direction is perpendicular to edge
- Gradient orientation is towards the higher brightness
- Gradient magnitude measures the edge strength

What is a 2D Gradient

so-called “nabla operator”

- the **gradient** is:

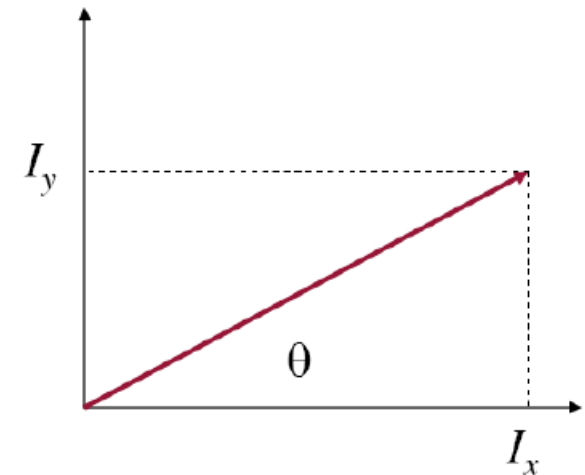
$$\nabla I = (I_x, I_y) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right)$$

- the **magnitude** of the gradient is:

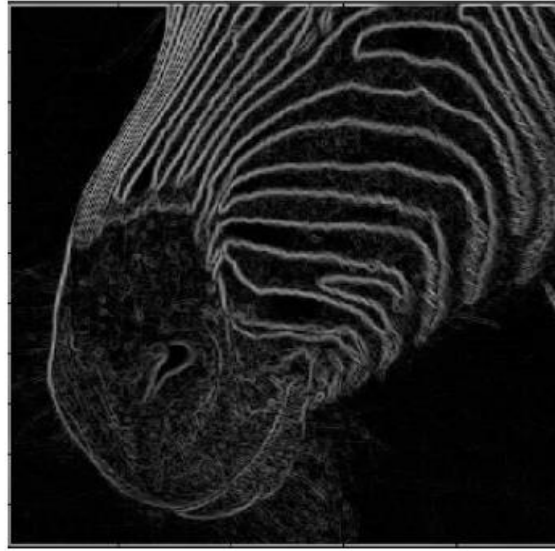
$$||\nabla I|| = \sqrt{I_x^2 + I_y^2}$$

- the **direction** of the gradient is:

$$\theta = \text{atan}(I_y, I_x)$$



Example – Gradient magnitude image

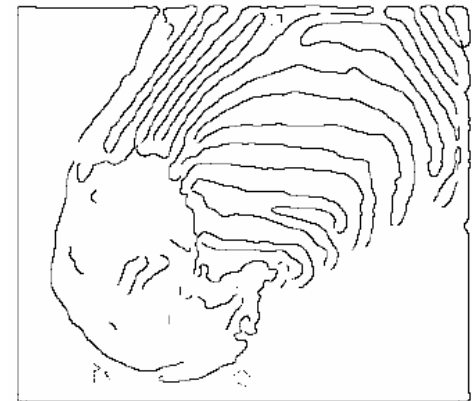


smoothed with
normal Gaussian



smoothed with
broad Gaussian

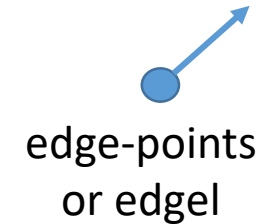
Our goal was to get thin edge chains...



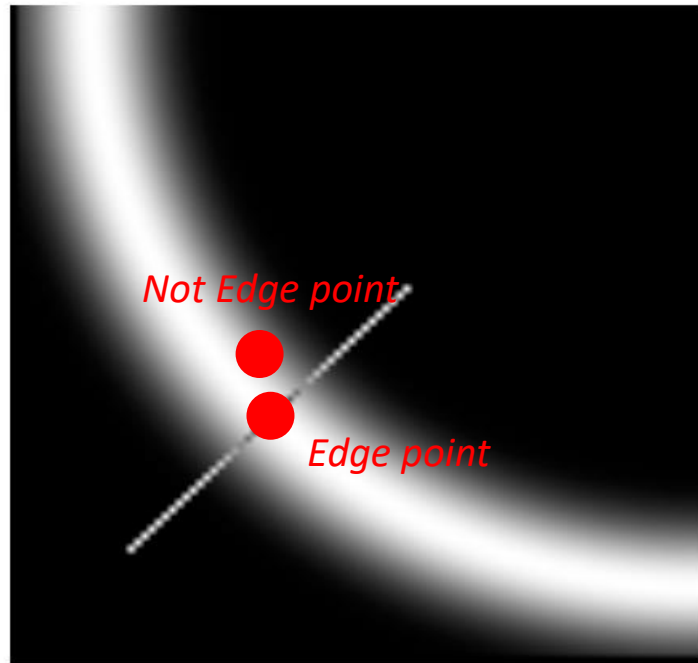
How to get edge chains

Outline of the widely-used Canny edge detector:

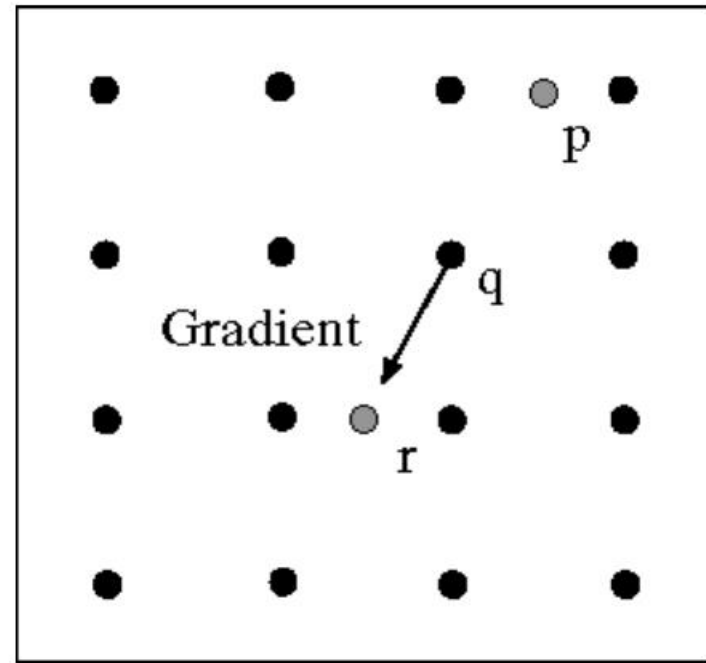
1. Compute robust Gradient Image: $((D_x * G) * I, (D_y * G) * I)$
2. Find edge-points (“edgels”): non-maximum suppression
3. Link-up edge-points to get chains
4. Do hysteresis to clean-up chains



Non-maximum suppression



Magnitude image M

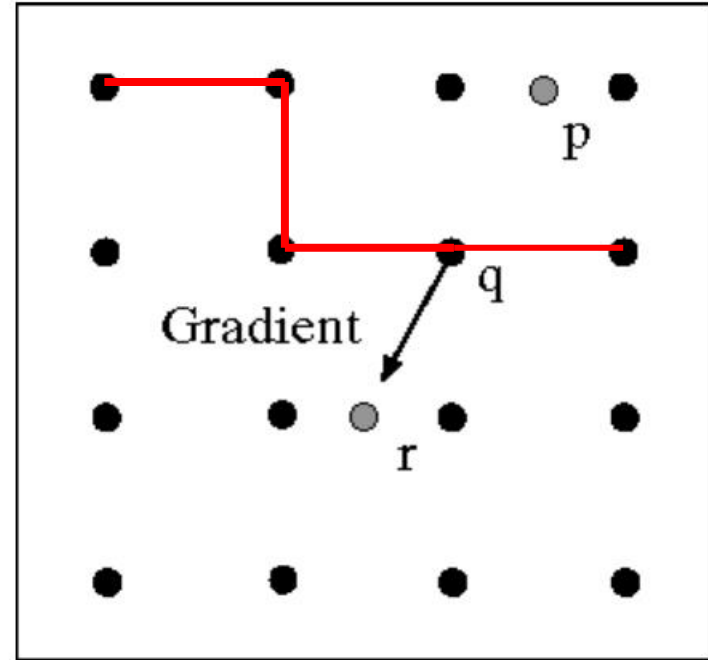
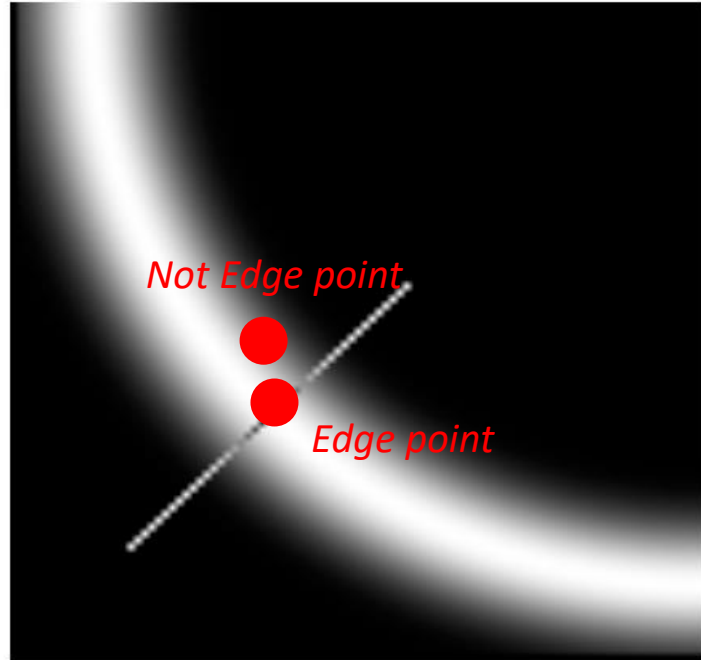


Zoom on pixel-grid

Which pixel is an edge point ? (non-max suppression)

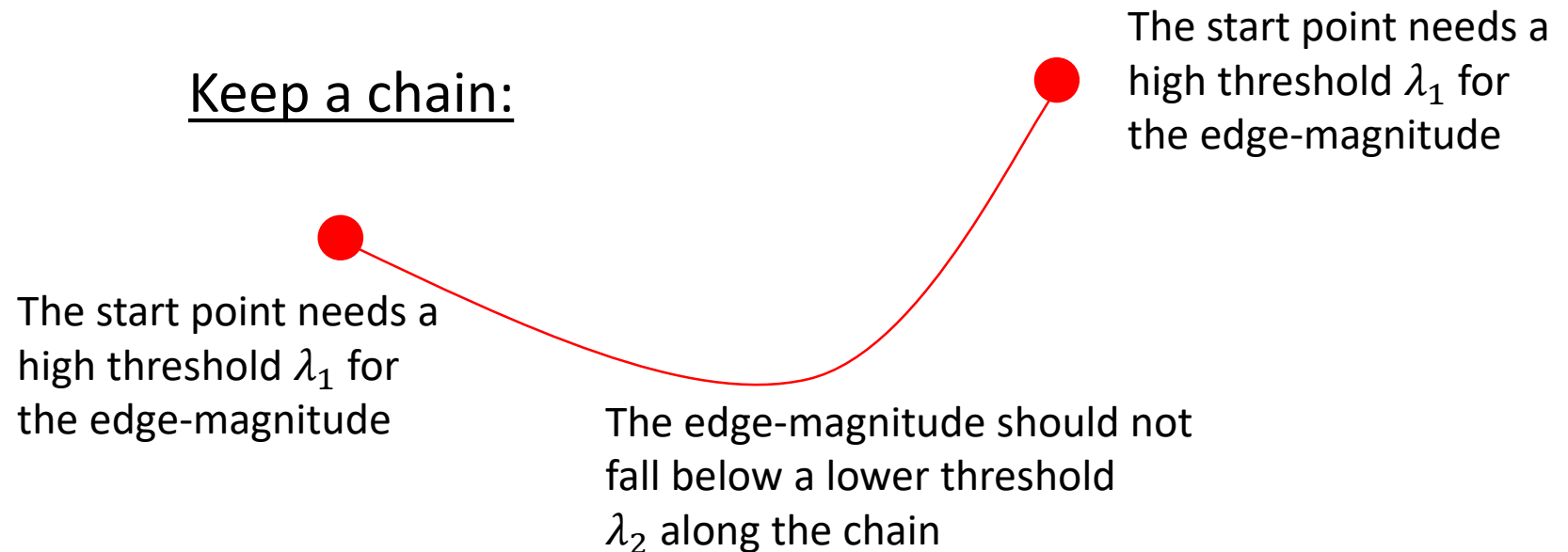
1. Check if pixel-value $M(q)$ is "local maximum" in "both" gradient directions, i.e. $M(q) \geq M(p)$ and $M(q) \geq M(r)$
2. Accept edge-point if above a threshold

Link up edge-points to get chains



Link-up neighbouring pixels if both are edge-points.

Clean up chains with Hysteresis

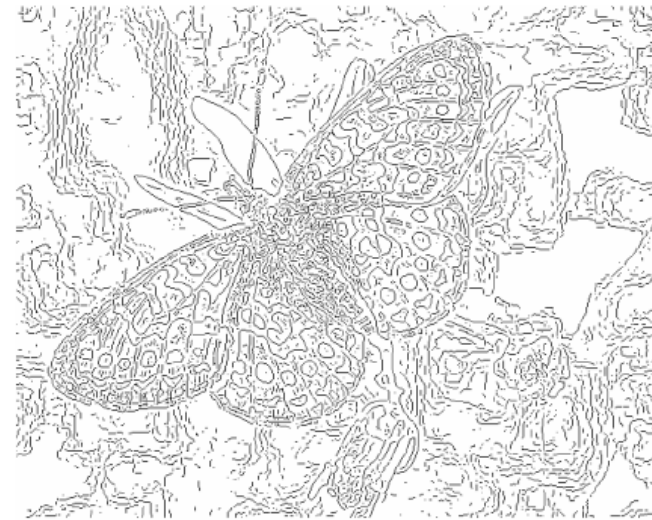


These two parameters are set for the whole image.

Final Result



Image



Not much smoothing (fine scale)



much smoothing (coarse scale)
small threshold



much smoothing (coarse scale)
large threshold

Usage



Original



Canny Edges
as guidance



Synthesized Image with
Stable Diffusion XL

ControlNet-XS: <https://vislearn.github.io/ControlNet-XS/>

Human-Level Edge Prediction

- So far we looked at “jumps” in gray-scale images?
- Humans perceive edges very differently (edges depend on semantic)



“average human drawing for
important edges”



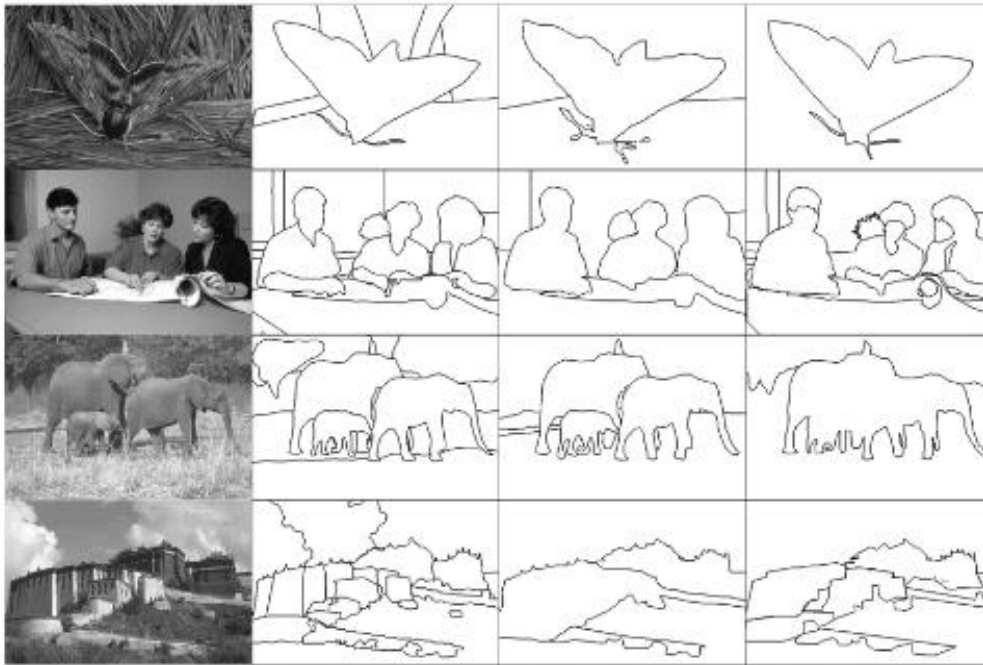
zoom

The computer vision algorithm has to reason
about semantics to find human-level edges

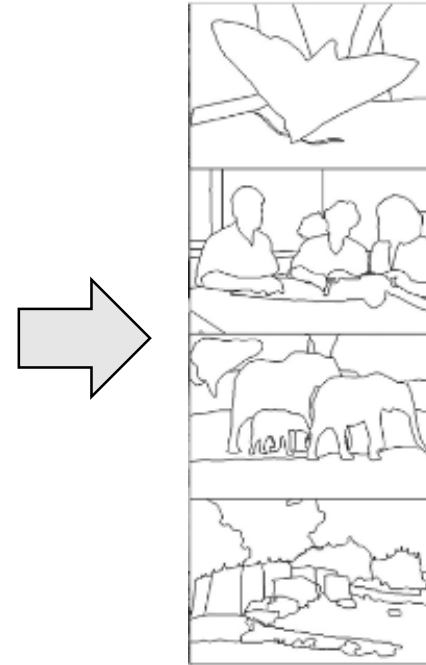
[Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues, Martin, Fowlkes and Malik PAMI 2004]

Human-Level Edge Prediction – BSDS Challenge

Instruction: “Divide each image into pieces, where each piece represents a distinguished thing in the image. It is important that all of the pieces have approximately equal importance. The number of things in each image is up to you. Something between 2 and 20 should be reasonable for any of our images”



Different human annotations



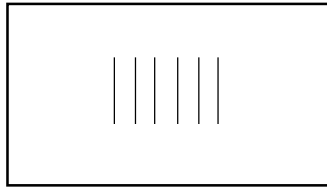
One true “consensus” edge map
(3 out of 5 humans agreed on an edge)

<https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/>

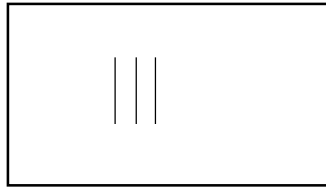
[P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik. Contour detection and hierarchical image segmentation. PAMI 2011]

[Martin et al., A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms and Measuring Ecological Statistics, ICCV 2001]

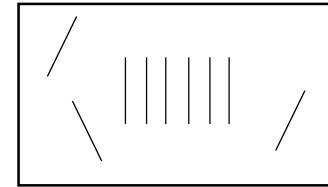
Question



Truth



Result A



Result B

Which result is better?

- 1) A
- 2) B
- 3) It depends on the application
- 4) I don't know

Answer: It depends on the application. Result B: in cancer research you do not want to miss a tumour; Result A: For movie suggestion in a home cinema you want to get only a few, most interesting movies suggested.

Precision and Recall

Details in [https://en.wikipedia.org/wiki/Precision_and_recall]

Definitions:

found items that are correct (true=correct; positive means detected)

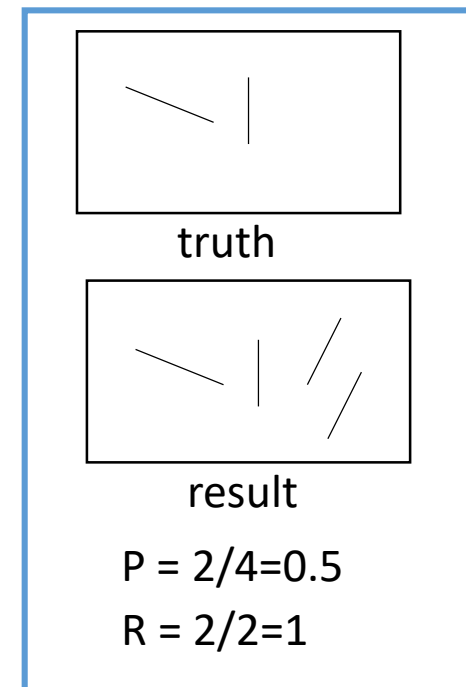
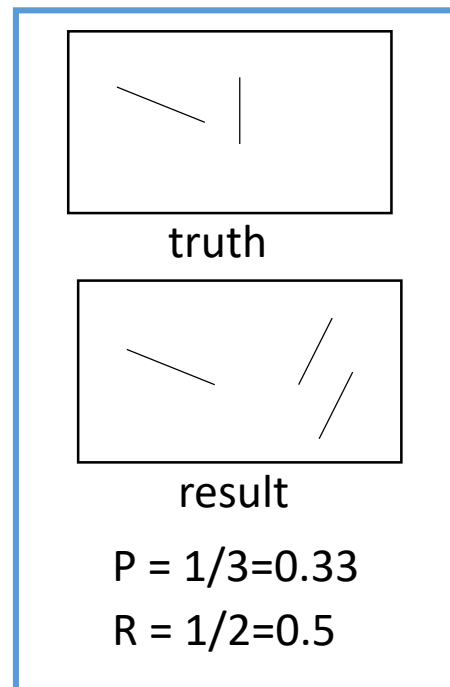
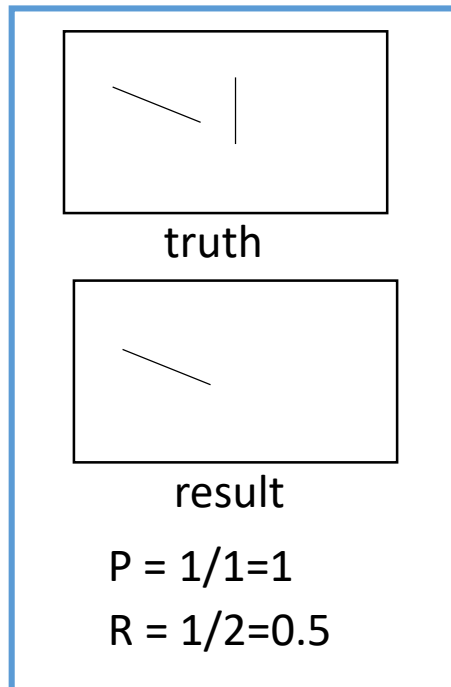
$$\text{Precision } P = \frac{\text{true positive}}{\text{number items in result}}$$

How many detected lines are correct?

$$\text{Recall } R = \frac{\text{true positive}}{\text{number items in truth}}$$

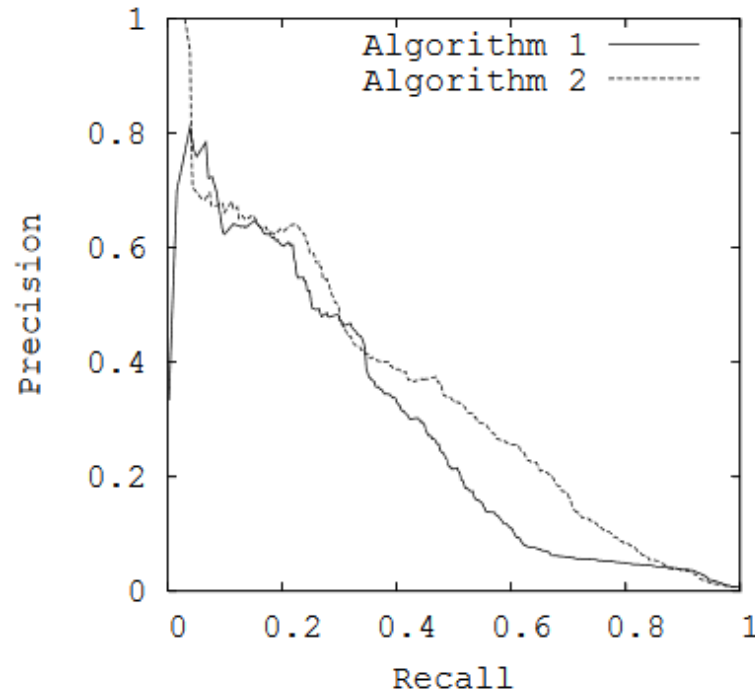
Did we find all lines?

Examples:



- Note, it is easy to get recall of 1, and also quite easy to get precision high

Precision-Recall Curve



- Common measure is Average Precision

$$AP = \int_0^1 p(r) dr \text{ where } p(r) \text{ is the precision for certain recall.}$$

- The best is to have $AP = 1$
- Example: We can systematically accept more and more lines and hence achieve finally a recall of 1.
- Note, ROC curve is different (true positive versus false positive)

F-score

What is the F Score?

The F score, also called the F1 score or F measure, is a measure of a test's accuracy. The F score is defined as the weighted **harmonic mean** of the test's **precision and recall**. This score is calculated according to:

$$F_1 = \left(\frac{\text{recall}^{-1} + \text{precision}^{-1}}{2} \right)^{-1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

with the precision and recall of a test taken into account. Precision, also called the positive predictive value, is the proportion of positive results that truly are positive. Recall, also called sensitivity, is the ability of a test to correctly identify positive results to get the true positive rate. The F score reaches the best value, meaning perfect precision and recall, at a value of 1. The worst F score, which means lowest precision and lowest recall, would be a value of 0.

Why is this Useful?

The F score is used to measure a test's accuracy, and it balances the use of precision and recall to do it. The F score can provide a more realistic measure of a test's performance by using both precision and recall. The F score is often used in information retrieval for measuring search, document classification, and query classification performance.

Harmonic Mean

$$H = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}} = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}} = \left(\frac{\sum_{i=1}^n x_i^{-1}}{n} \right)^{-1}$$

Since the harmonic mean of a list of numbers tends strongly toward the least elements of the list, it tends (compared to the arithmetic mean) to mitigate the impact of large outliers and increase the impact of small ones.

Precision-Recall Curve for BSDS Challenge

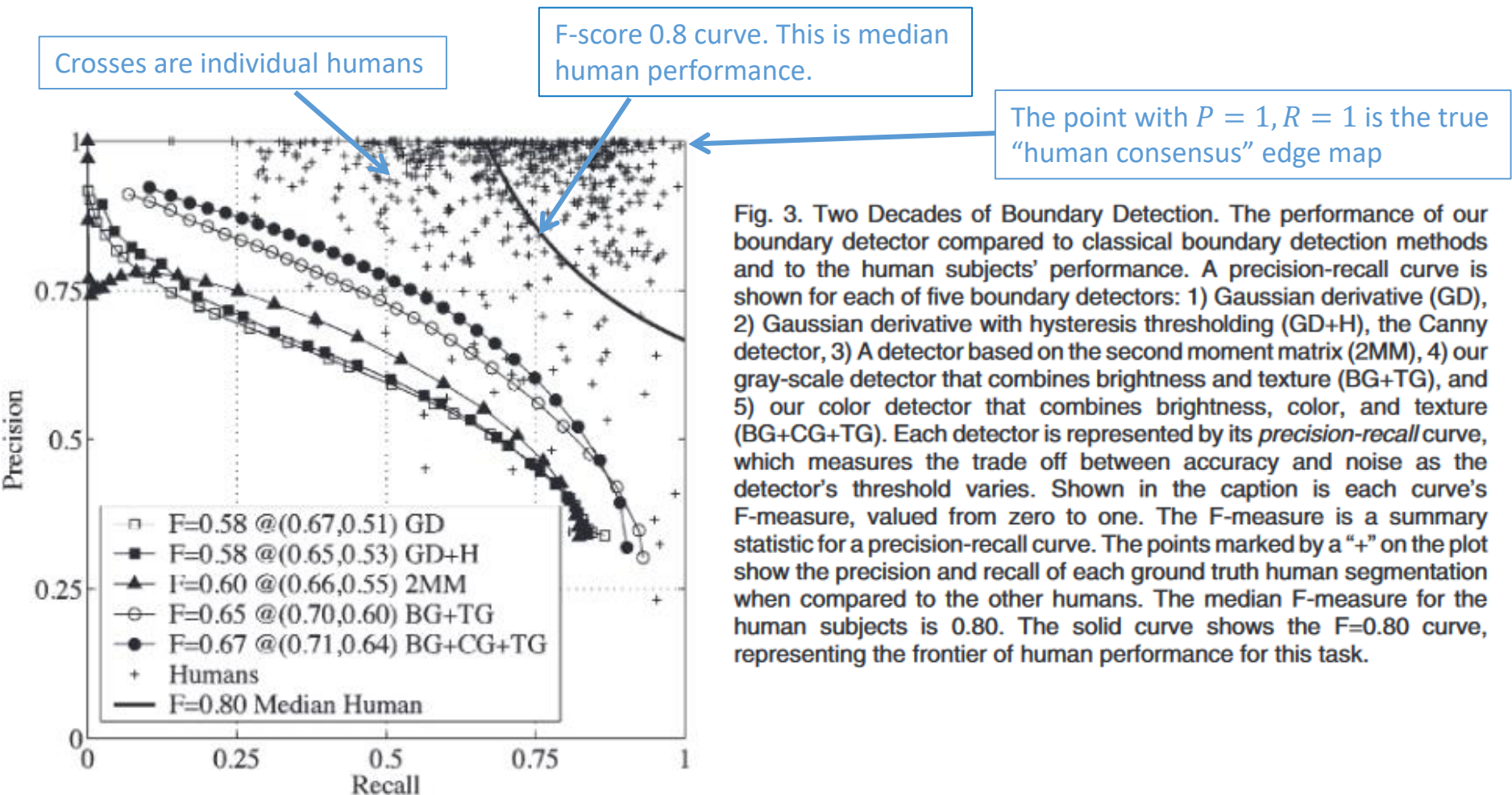


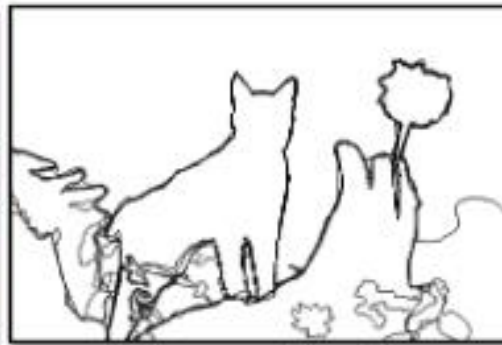
Fig. 3. Two Decades of Boundary Detection. The performance of our boundary detector compared to classical boundary detection methods and to the human subjects' performance. A precision-recall curve is shown for each of five boundary detectors: 1) Gaussian derivative (GD), 2) Gaussian derivative with hysteresis thresholding (GD+H), the Canny detector, 3) A detector based on the second moment matrix (2MM), 4) our gray-scale detector that combines brightness and texture (BG+TG), and 5) our color detector that combines brightness, color, and texture (BG+CG+TG). Each detector is represented by its *precision-recall* curve, which measures the trade off between accuracy and noise as the detector's threshold varies. Shown in the caption is each curve's F-measure, valued from zero to one. The F-measure is a summary statistic for a precision-recall curve. The points marked by a "+" on the plot show the precision and recall of each ground truth human segmentation when compared to the other humans. The median F-measure for the human subjects is 0.80. The solid curve shows the F=0.80 curve, representing the frontier of human performance for this task.

[Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues, Martin, Fowlkes and Malik PAMI 2004]

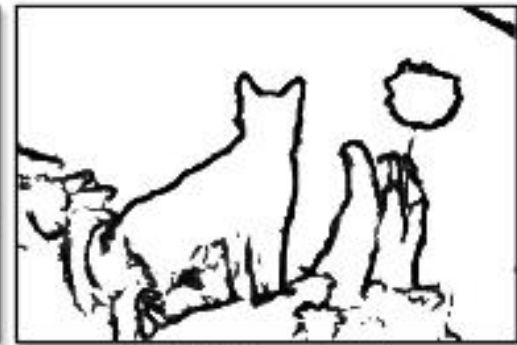
Learn semantically meaningful edges



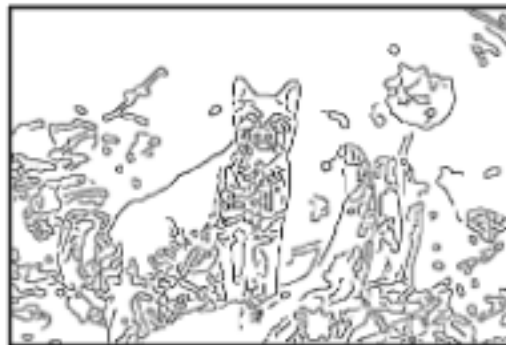
(a) original image



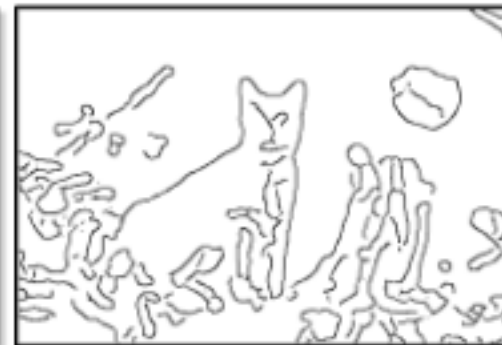
(b) ground truth



(c) HED: output
(Holistically-Nested Edge Detection)



(g) Canny: $\sigma = 2$



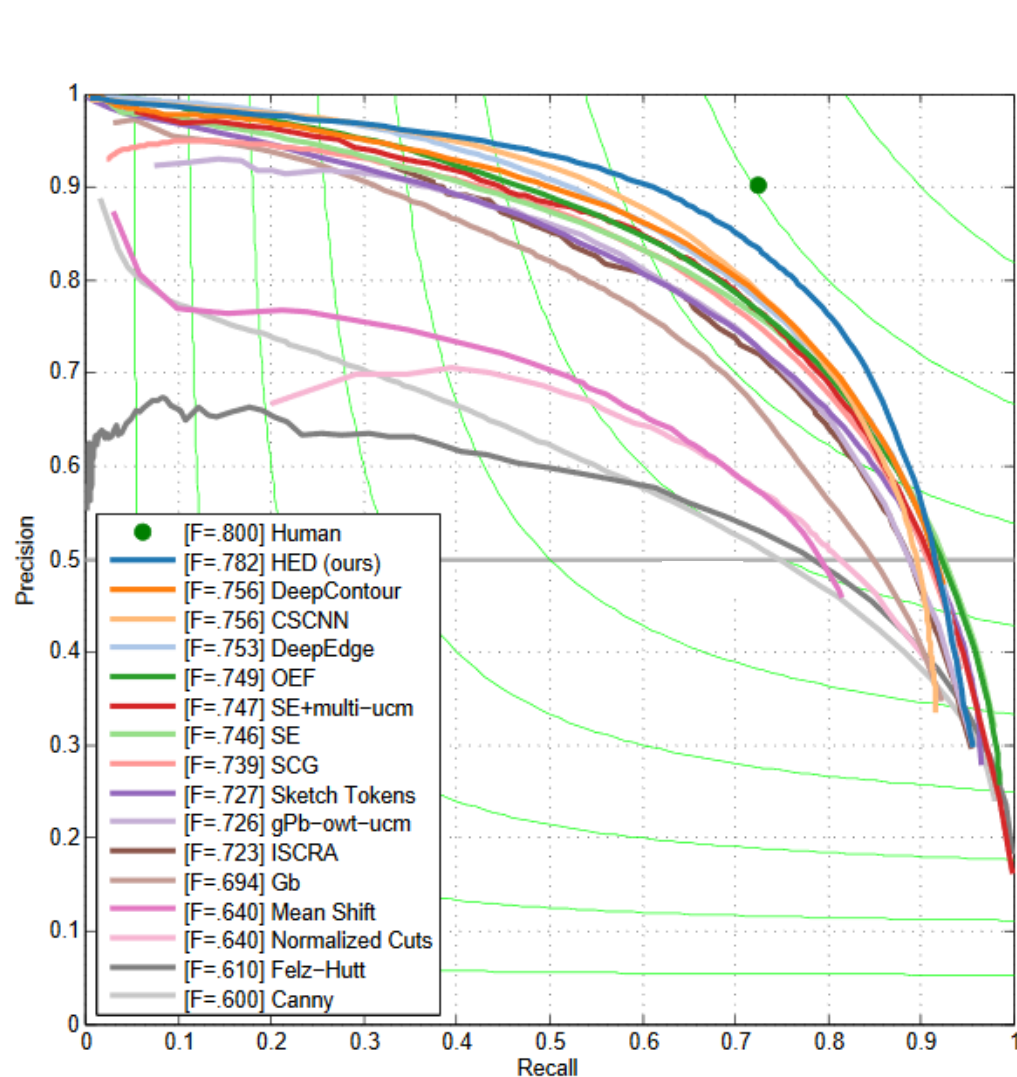
(h) Canny: $\sigma = 4$



(i) Canny: $\sigma = 8$

[Holistically-Nested Edge Detection, Xie, Tu ICCV 2015]

Learn semantically meaningful edges

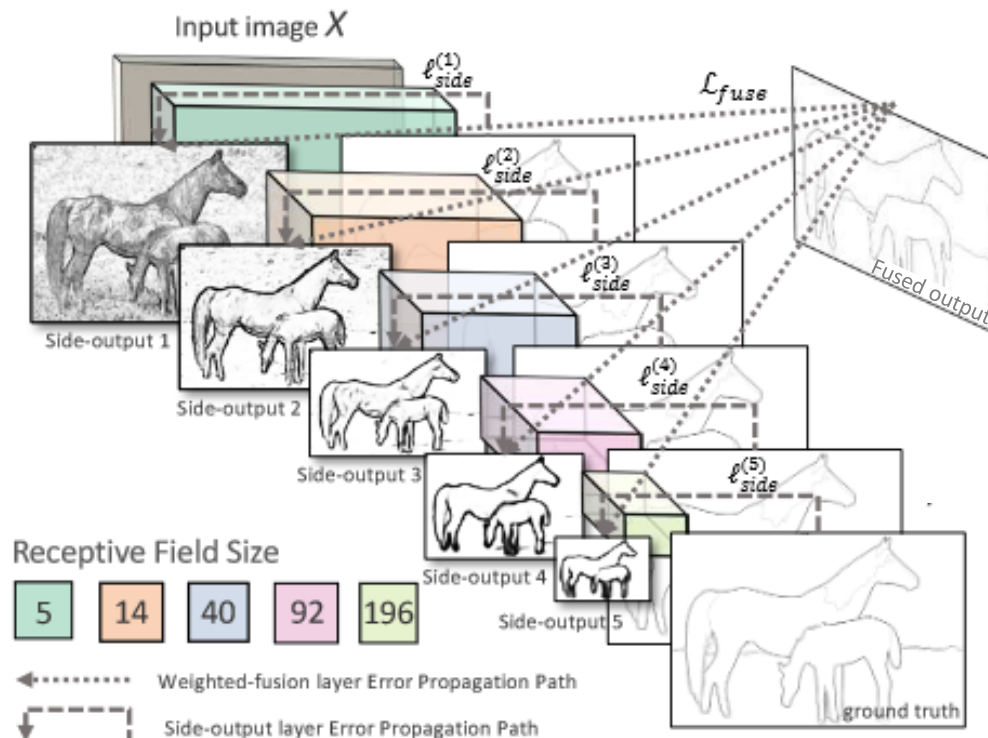


Green lines are F-score isolines

	(Frames per second)	
	AP	FPS
Human	-	-
Canny	.580	15
Felz-Hutt [9]	.560	10
BEL [5]	-	1/10
gPb-owt-ucm [1]	.696	1/240
Sketch Tokens [24]	.780	1
SCG [31]	.773	1/280
SE-Var [6]	.803	2.5
OEF [13]	.817	-
DeepNets [21]	.758	1/5 [†]
N4-Fields [10]	.784	1/6 [†]
DeepEdge [2]	.807	1/10 ^{3†}
CSCNN [19]	.798	-
DeepContour [34]	.797	1/30 [†]
HED (ours)	.833	2.5[†], 1/12

[Holistically-Nested Edge Detection, Xie, Tu ICCV 2015]

Learn semantically meaningful edges



Main property: estimating edges at different image scales produces different types of edges.

Testing:

$$(\hat{Y}_{fuse}, \hat{Y}_{side}^{(1)}, \dots, \hat{Y}_{side}^{(M)}) = \text{CNN}(X, (\mathbf{W}, \mathbf{w}, \mathbf{h})^*),$$

$$\hat{Y}_{HED} = \text{Average}(\hat{Y}_{fuse}, \hat{Y}_{side}^{(1)}, \dots, \hat{Y}_{side}^{(M)})$$

- with $\hat{A}_{side}^{(m)} \equiv \{a_j^{(m)}, j = 1, \dots, |Y|\}$ are activations of the side-output of layer m
- and $\hat{Y}_{side}^{(m)} = \sigma(\hat{A}_{side}^{(m)})$.
- and $\hat{Y}_{fuse} \equiv \sigma(\sum_{m=1}^M h_m \hat{A}_{side}^{(m)})$
- with σ is the sigmoid function

- Input image X
- Standard Network layer weights $\mathbf{W}$
- Side output layers $\mathbf{w} = (\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(M)})$
- Fusion weights $\mathbf{h} = (h_1, \dots, h_M)$

Figure 3. Illustration of our network architecture for edge detection, highlighting the error backpropagation paths. Side-output layers are inserted after convolutional layers. Deep supervision is imposed at each side-output layer, guiding the side-outputs towards edge predictions with the characteristics we desire. The outputs of HED are multi-scale and multi-level, with the side-output-plane size becoming smaller and the receptive field size becoming larger. One weighted-fusion layer is added to automatically learn how to combine outputs from multiple scales. The entire network is trained with multiple error propagation paths (dashed lines).

Learn semantically meaningful edges

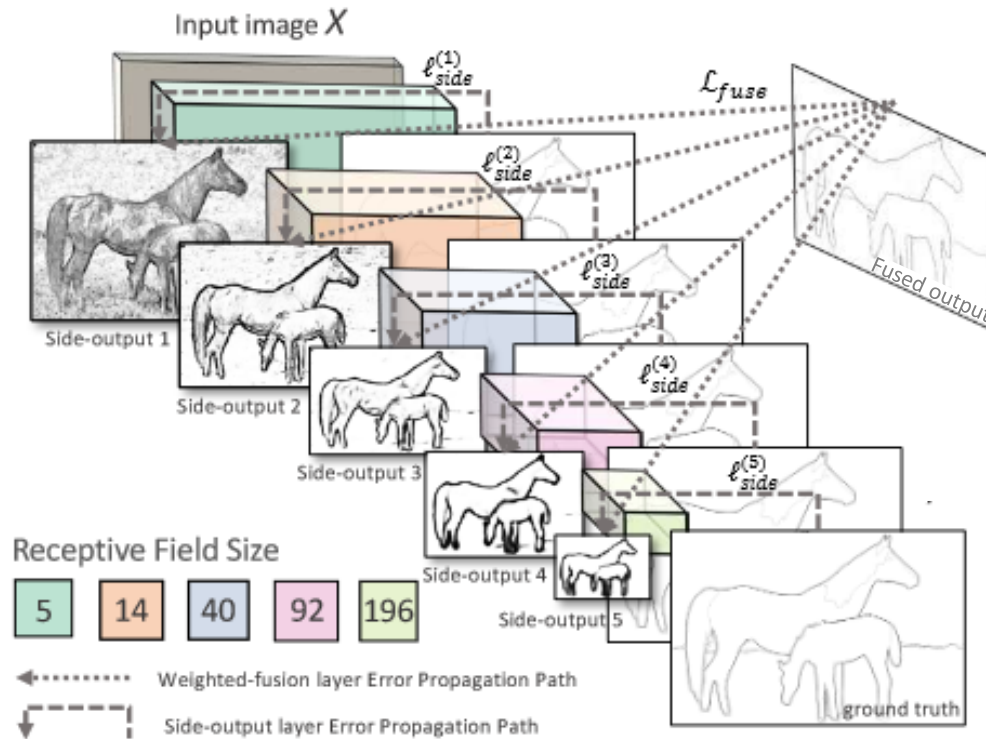


Figure 3. Illustration of our network architecture for edge detection, highlighting the error backpropagation paths. Side-output layers are inserted after convolutional layers. Deep supervision is imposed at each side-output layer, guiding the side-outputs towards edge predictions with the characteristics we desire. The outputs of HED are multi-scale and multi-level, with the side-output-plane size becoming smaller and the receptive field size becoming larger. One weighted-fusion layer is added to automatically learn how to combine outputs from multiple scales. The entire network is trained with multiple propagation paths (dashed lines).

Main property: estimating edges at different image scales produces different types of edges.

Training:

$$(\mathbf{W}, \mathbf{w}, \mathbf{h})^* = \operatorname{argmin}(\mathcal{L}_{\text{side}}(\mathbf{W}, \mathbf{w}) + \mathcal{L}_{\text{fuse}}(\mathbf{W}, \mathbf{w}, \mathbf{h}))$$

$$\mathcal{L}_{\text{fuse}}(\mathbf{W}, \mathbf{w}, \mathbf{h}) = \text{Dist}(Y, \hat{Y}_{\text{fuse}})$$

$$\begin{aligned} \ell_{\text{side}}^{(m)}(\mathbf{W}, \mathbf{w}^{(m)}) = & -\beta \sum_{j \in Y_+} \log \Pr(y_j = 1 | X; \mathbf{W}, \mathbf{w}^{(m)}) \\ & - (1 - \beta) \sum_{j \in Y_-} \log \Pr(y_j = 0 | X; \mathbf{W}, \mathbf{w}^{(m)}) \end{aligned}$$

- Ground Truth is Y
- Y_- are GT-edges (black)
- Y_+ are GT-non-edges (white)
- $\ell_{\text{side}}^{(m)}$ is weighted cross entry loss
- class-weighting β :
 - $\beta = \frac{|Y_-|}{|Y|}$ (typically small)
 - $(1 - \beta) = \frac{|Y_+|}{|Y|}$ (typically large)

Recent Work ... line of research comes to an end

BSDS Dataset is not perfect:

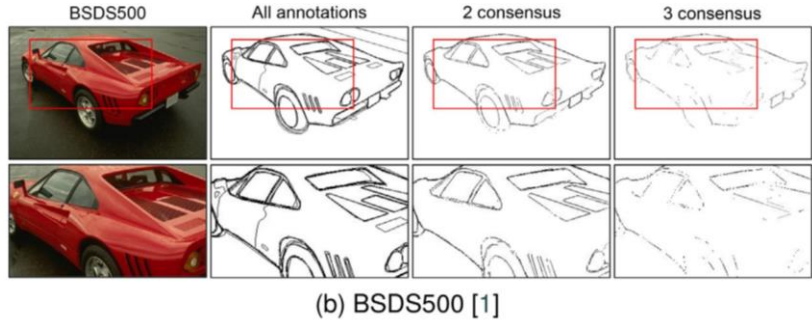


Fig. 2. A careful examination of the BSDS benchmark datasets for two sample images and their corresponding ground truths. There is a large discrepancy among human-drawn “edges”. Enforcing consensus among two/three annotations results in the elimination of true edges. This issue demonstrates that BSDS is a suitable dataset for evaluating boundaries but not edges.

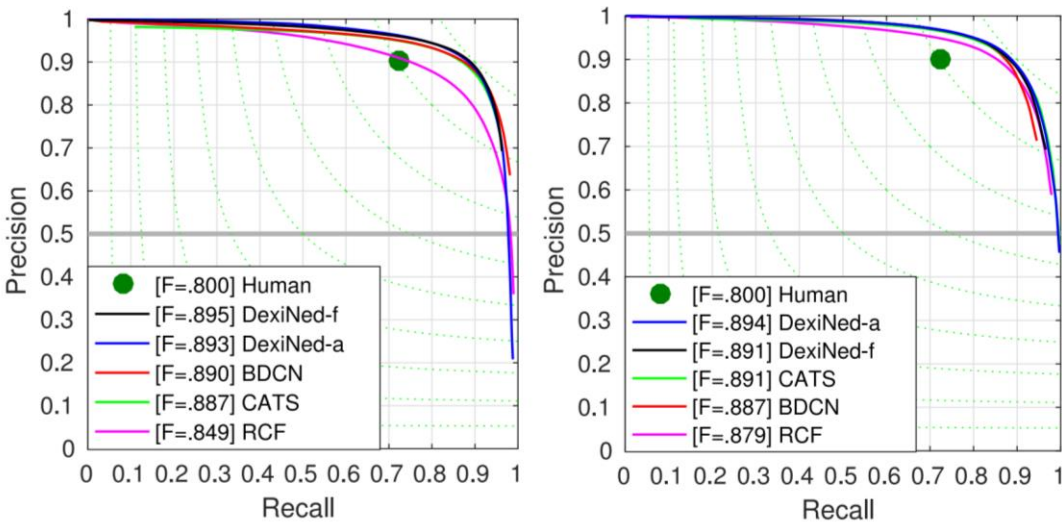


Fig. 7. (left) Precision-Recall curve of BIPED test set. (right) Precision-Recall curve of MDBD [48].

[Dense extreme inception network for edge detection; X. Soria et al. Pattern Recognition, 2023]

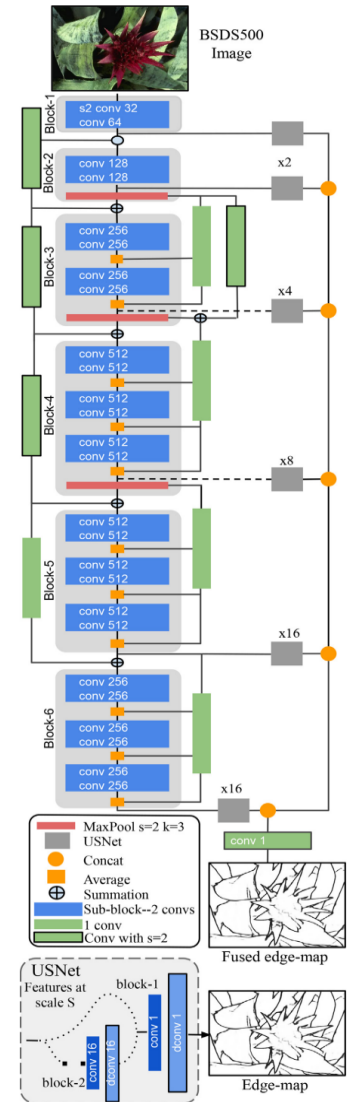


Fig. 3. Flowchart of proposed architecture (DexiNed). It consists of two building blocks a Dense Extreme Inception Network and an upsampling Net (USNet).

Recent Work ... line of research comes to an end



Fig. 8. The results of a few state-of-the-art architectures trained on the proposed BIPED dataset. Note that the ground truth of these datasets, except for BIPED and MDBD, correspond to boundary and segmentation tasks.

[Dense extreme inception network for edge detection; X. Soria et al. Pattern Recognition, 2023]

Roadmap: Basics of Digital Image Processing

- What is an Image?
- Point operators (ch. 3.1)
- Filtering: (ch. 3.2, ch 3.3, ch. 3.4)
 - Linear filtering
 - Non-linear filtering
- Real-world Denoising
- Multi-scale image representation (ch. 3.5)
- Edges detection and linking (ch. 4.2)
- Interest Point detection (ch. 4.1.1)

- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2011. (First Edition)
An earlier version of the book is online: <http://szeliski.org/Book/>
- Computer Vision: Algorithms and Applications by Rick Szeliski; Springer 2021. (Second Edition)
https://www.dropbox.com/s/8bf4feleifhrvl6/SzeliskiBookDraft_20210930.pdf?dl=0

Question



Patches taken:



A

B

C



Patches taken:



A'

B'

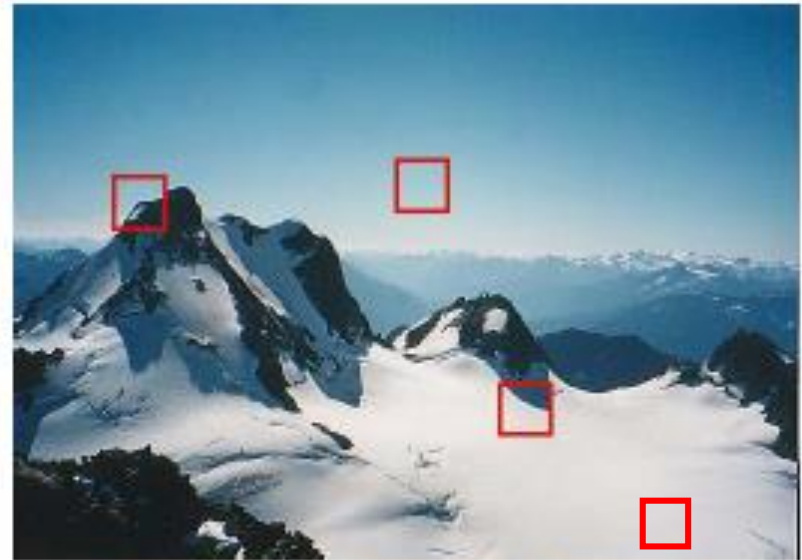
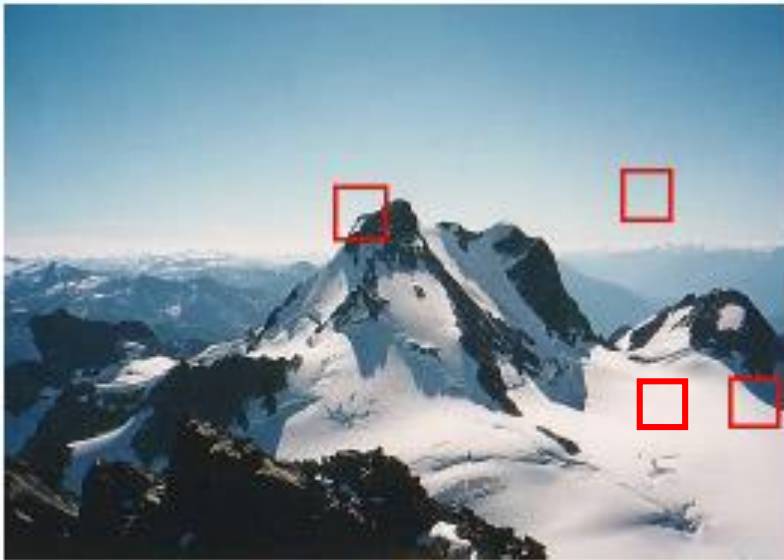
C'

For which pair of patches are you sure that they show **the same** 3D scene?

- 1) (A, A')
- 2) (B, B')
- 3) (C, C')
- 4) I don't know

What region should we try to match?

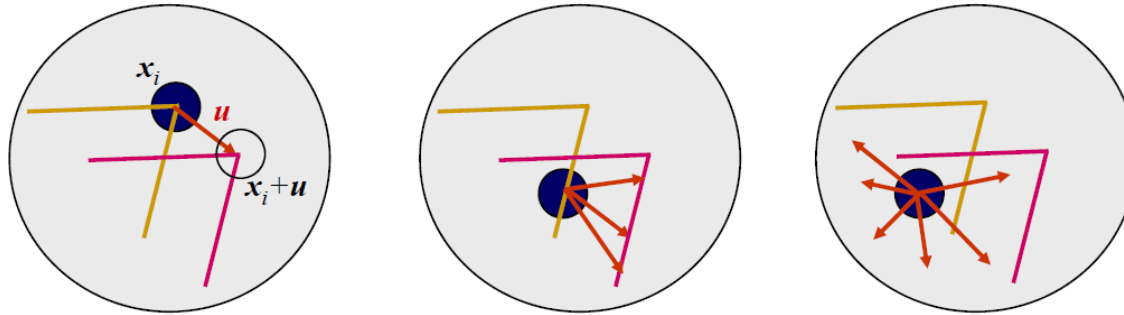
We want to find a few regions where this image pair matches: Applications later



Look for a region that is **unique**, i.e. not ambiguous

Points versus Lines

„Aperture problem“

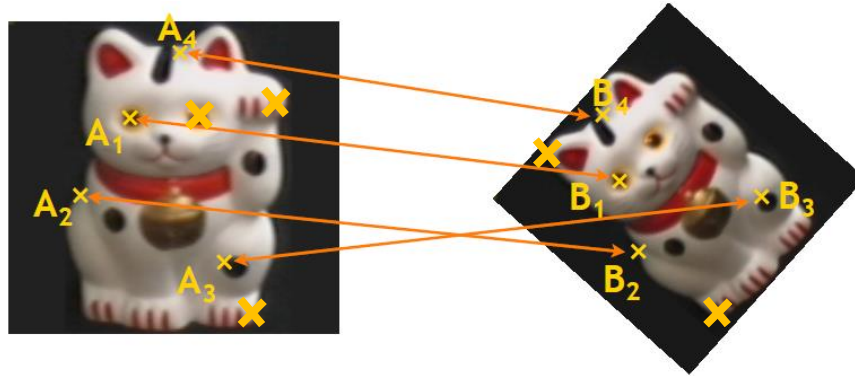


Lines are not as good as points



Goal: Interest Point Detection

- Goal: predict a few “**interest points**” in order to remove redundant data efficiently



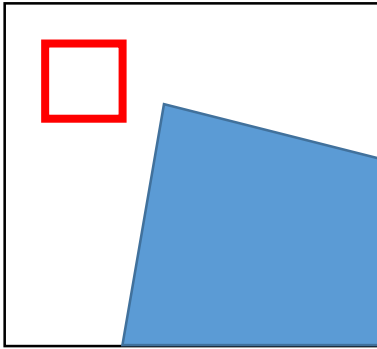
Should be invariant against:

- a. Geometric transformation – scaling, rotation, translation, affine transformation, projective transformation etc.
- b. Color transformation – additive (lightning change), multiplicative (contrast), linear (both), monotone etc.;
- c. Discretization (e.g. spatial resolution, focus);

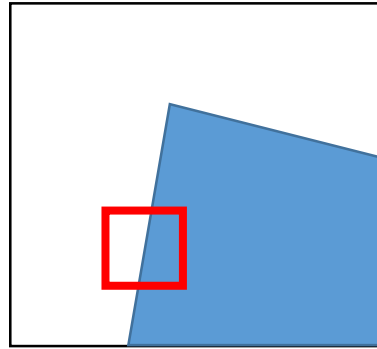
Harris Detector

Local measure of **feature uniqueness**:

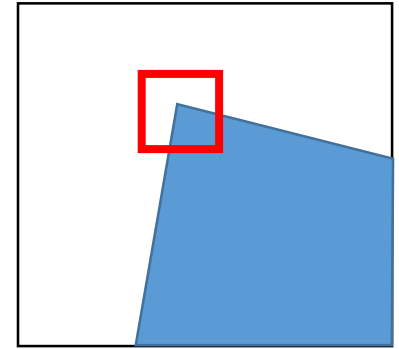
Shifting the window in any direction: how does it change?



“flat” region:
no change in all
directions



“edge”:
no change along the
edge direction



“corner”:
significant change in
all directions

[Szeliski and Seitz]

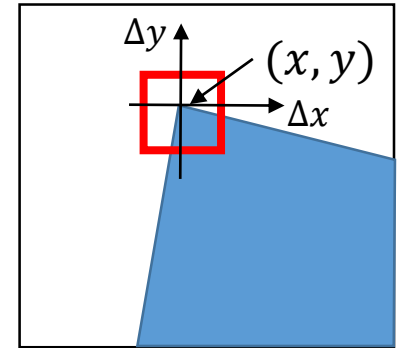
Harris Detector

Harris Detector

How similar is the image $I(x, y)$ to itself?

Autocorrelation function:

$$c(x, y, \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} w(u, v) \left(I(u, v) - I(u + \Delta x, v + \Delta y) \right)^2$$



$W(x, y)$ is a small window around (x, y)

$w(u, v)$ is a convolution kernel and used to decrease the influence of pixels far from (x, y) , e.g. the Gaussian $\exp \left[-\frac{(u-x)^2 + (v-y)^2}{2\sigma^2} \right]$.

For simplicity we use $w(u, v) = 1$

Harris Detector

$$c(x, y, \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} \left(I(u, v) - I(u + \Delta x, v + \Delta y) \right)^2$$

We are interested in **properties** of $c(x, y, \Delta x, \Delta y)$ at a specific position (x, y)

Let us look at a linear approximation of $I(u + \Delta x, v + \Delta y)$.

Taylor expansion around (u, v)

$$\begin{aligned} I(u + \Delta x, v + \Delta y) &= I(u, v) + \frac{\partial I(u, v)}{\partial x} \Delta x + \frac{\partial I(u, v)}{\partial y} \Delta y + \epsilon(\Delta x, \Delta y) \\ &\approx I(u, v) + \underbrace{[I_x(u, v), I_y(u, v)]}_{\text{Gradient at } (u, v)} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \end{aligned}$$

Reminder Taylor expansion around a of function $f(b + x)$:

$$f(b + x) = f(a) + \frac{\partial f(a)}{\partial x} (b + x - a) + \frac{\partial^2 f(a)}{\partial x^2} (b + x - a)^2 + \dots$$

Harris Detector

Put it together:

$$\begin{aligned}c(x, y, \Delta x, \Delta y) &= \sum_{(u,v) \in W(x,y)} \left(I(u, v) - I(u+\Delta x, v+\Delta y) \right)^2 \\&\approx \sum_{(u,v) \in W(x,y)} \left([I_x(u, v), I_y(u, v)] \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \right)^2 \\&= [\Delta x, \Delta y] Q(x, y) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}\end{aligned}$$

Q is called the Structure Tensor

with

$$Q(x, y) = \begin{bmatrix} \sum_W I_x(u, v)^2 & \sum_W I_x(u, v) I_y(u, v) \\ \sum_W I_x(u, v) I_y(u, v) & \sum_W I_y(u, v)^2 \end{bmatrix} = \begin{bmatrix} A & B \\ B & C \end{bmatrix}$$

We compute this at any image location (x, y)

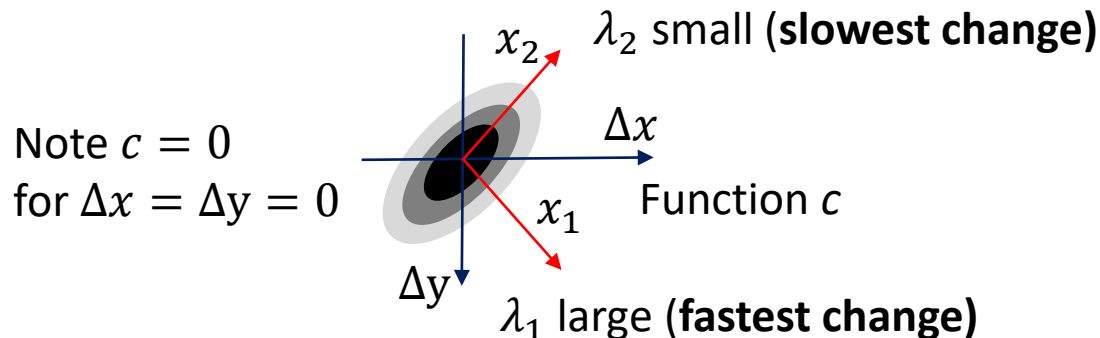
Harris Detector

The autocorrelation function

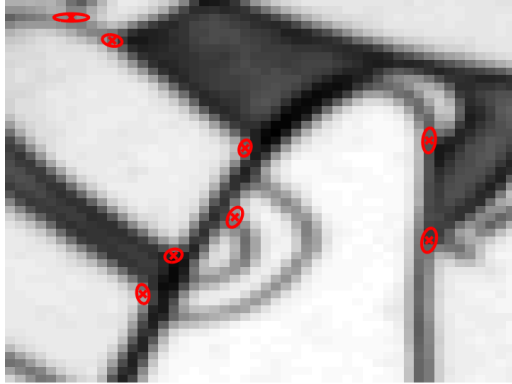
$$c(x, y, \Delta x, \Delta y) \approx [\Delta x, \Delta y] Q(x, y) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

Function c is (with approximation) a **quadratic** function in Δx and Δy

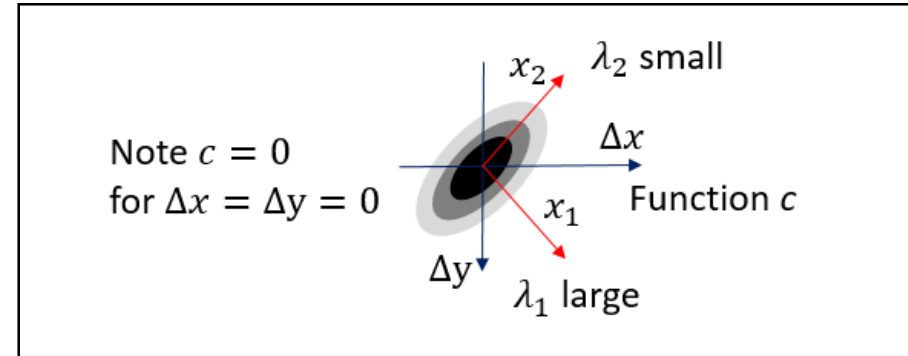
- Isolines are ellipses ($Q(x, y)$ is symmetric and positive definite);
- Eigenvector x_1 with (larger) Eigenvalue λ_1 is the direction of **fastest change** in function c
- Eigenvector x_2 with (smaller) Eigenvalue λ_2 is direction of **slowest change** in function c



Question



Corners are shown in red

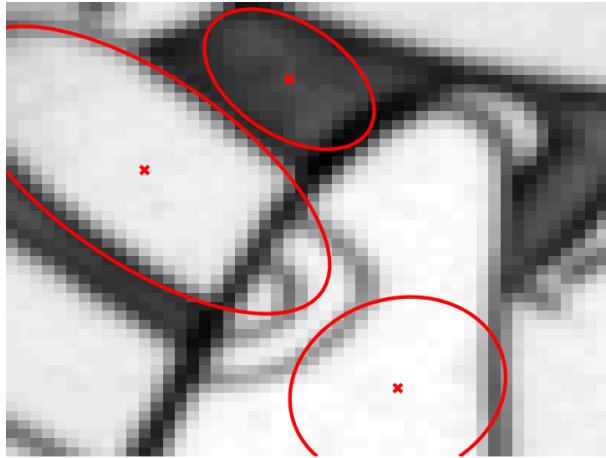


What are the eigenvalues at corners:

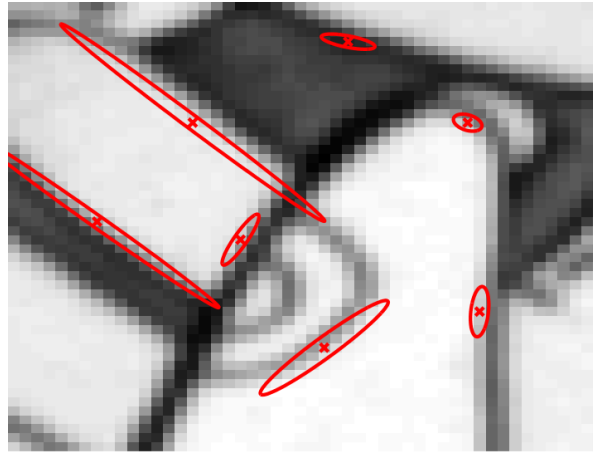
- 1) λ_1 small, λ_2 small?
- 2) λ_1 small, λ_2 large?
- 3) λ_1 large, λ_2 small?
- 4) λ_1 large, λ_2 large?
- 5) I don't know

Harris Detector

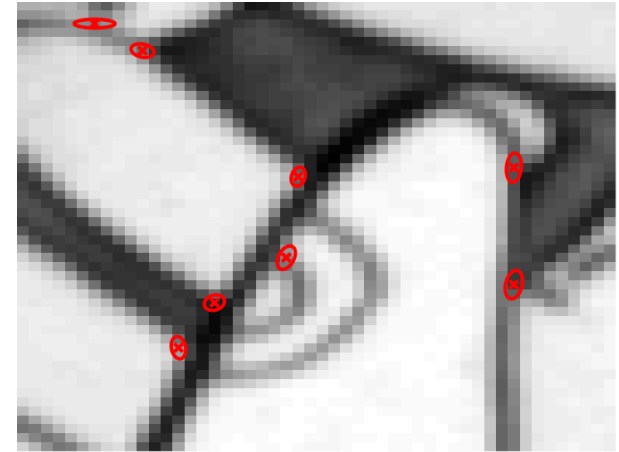
Some examples – isolines for $c(x, y, \Delta x, \Delta y) = 1$ with fixed (x, y) and variable $(\Delta x, \Delta y)$



(a) Flat



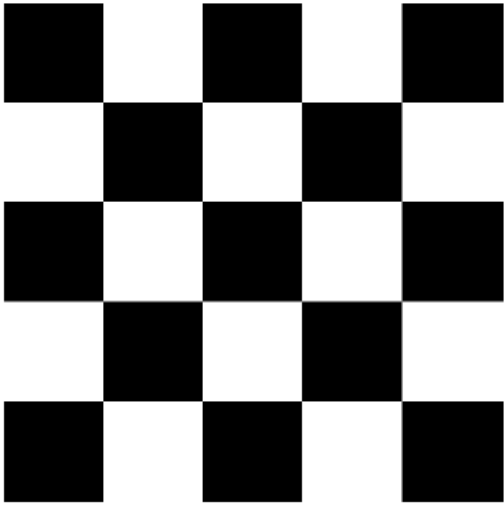
(b) Edges



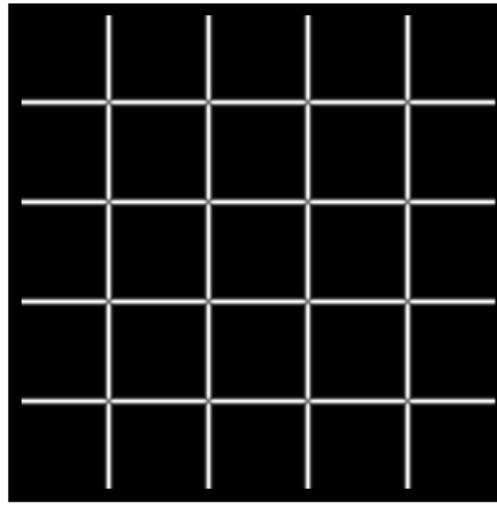
(c) Corners

- a. Homogenous regions: λ_1 and λ_2 are small
- b. Edges: λ_1 is large and λ_2 is small
- c. Corners: λ_1 and λ_2 are large (this is what we are looking for!)

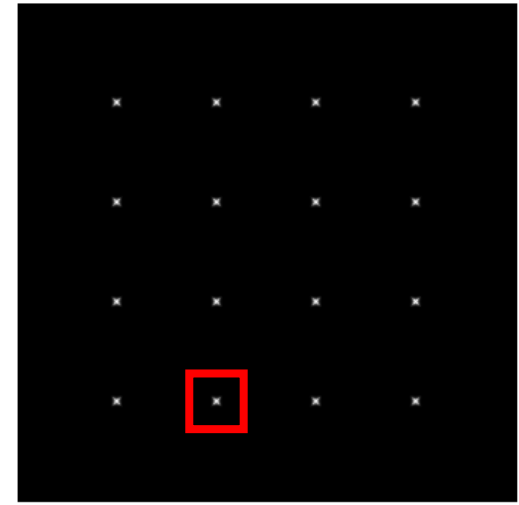
Harris Detector



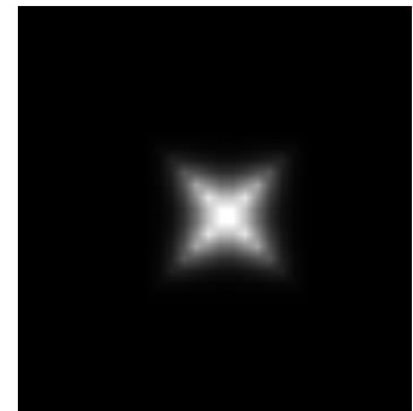
Image



λ_1 (larger eigenvalue)



λ_2 (smaller eigenvalue)



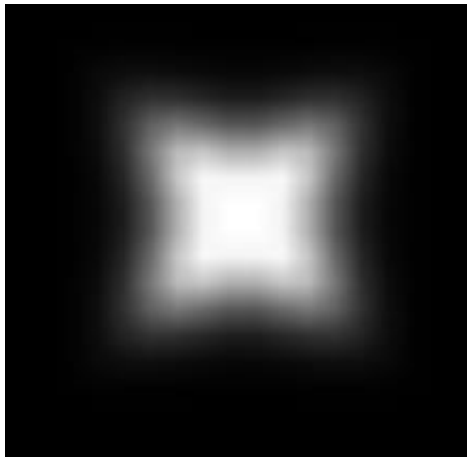
Zoom in

Harris Detector

“**Cornerness**” is a characteristic of $Q(x, y)$

$$\lambda_1 \lambda_2 = \det Q(x, y) = AC - B^2, \quad \lambda_1 + \lambda_2 = \text{trace} Q(x, y) = A + C$$

Proposition by Harris: $H = \lambda_1 \lambda_2 - 0.04(\lambda_1 + \lambda_2)^2$

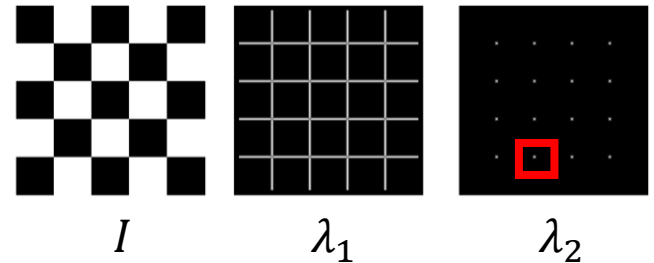


Harris Value



Smallest eigenvalue

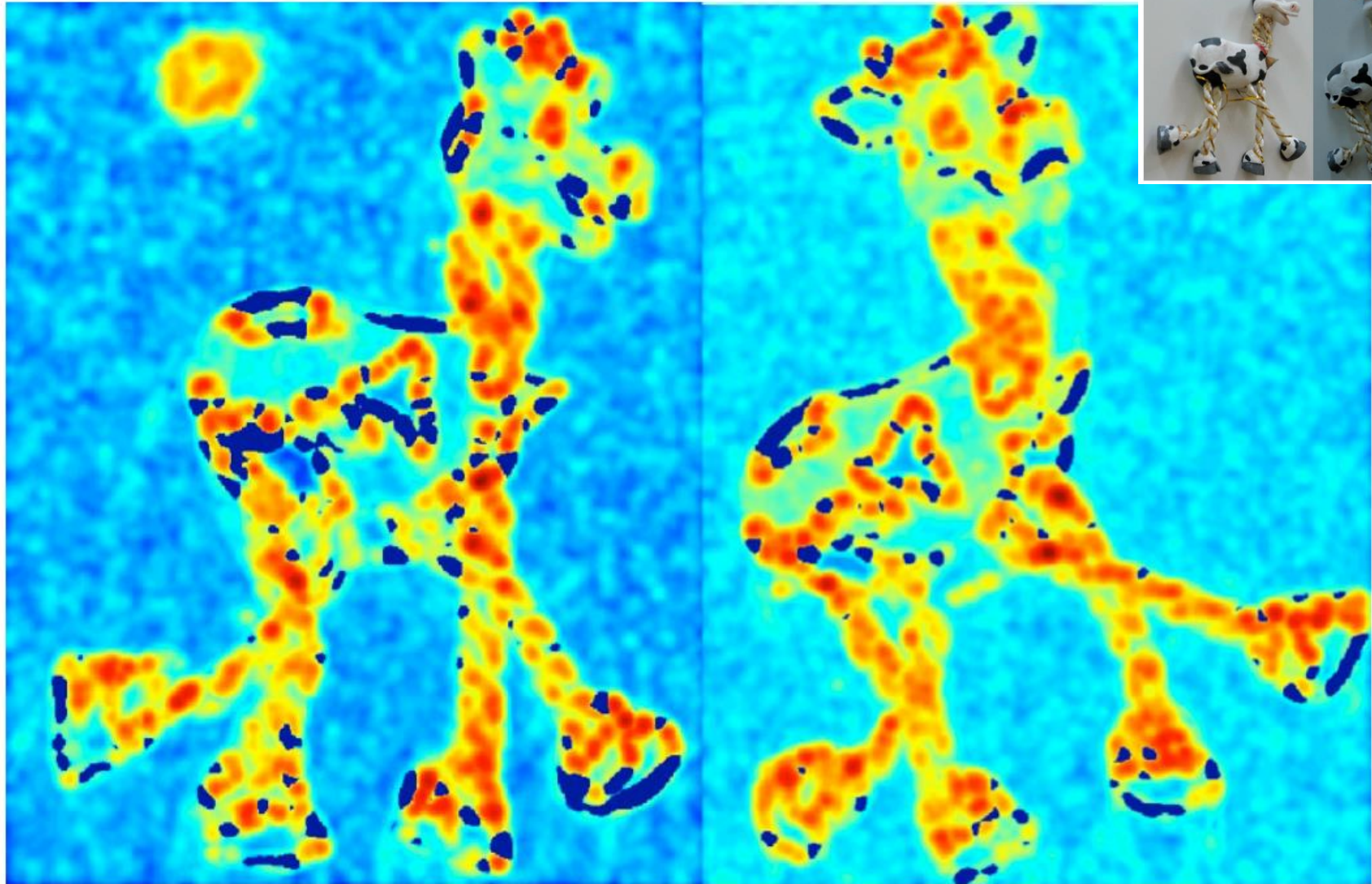
(note, many local minima
close to largest peak)



Harris Corners - example



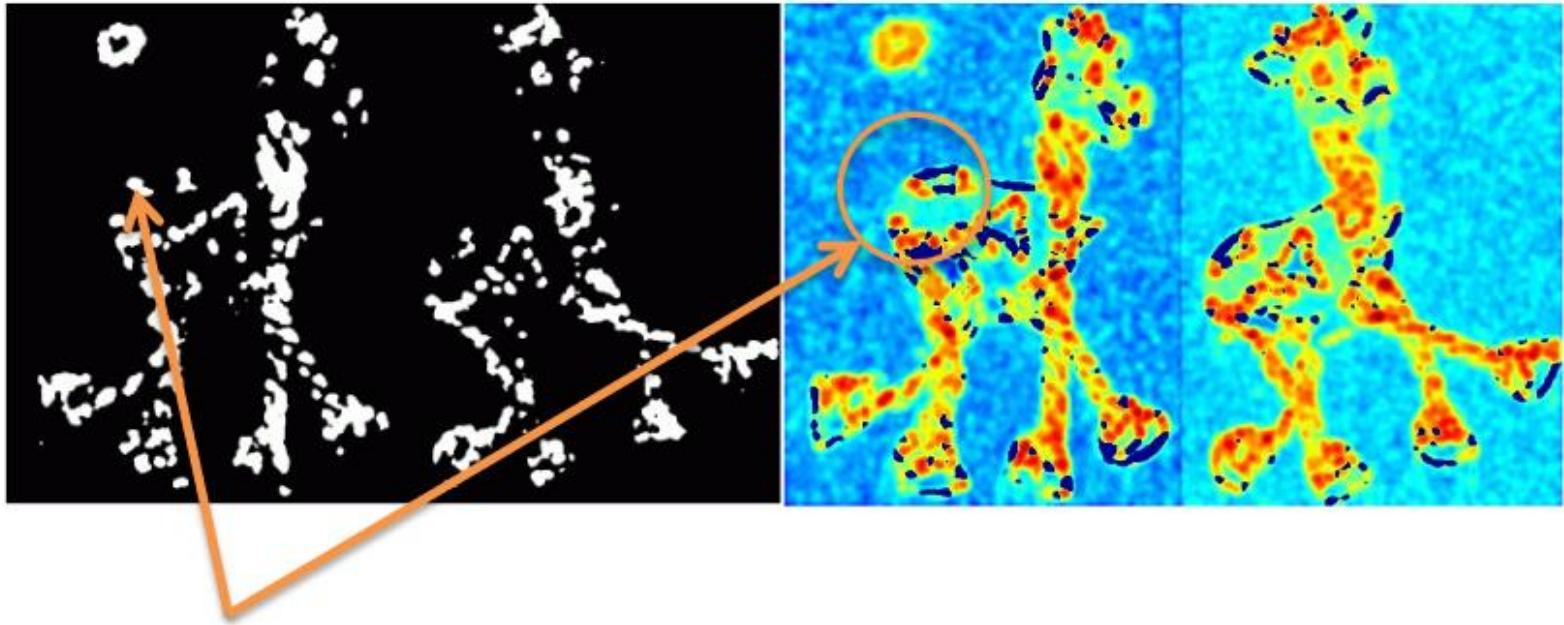
H-score (red- high, blue - low)



Threshold (H-score > value)



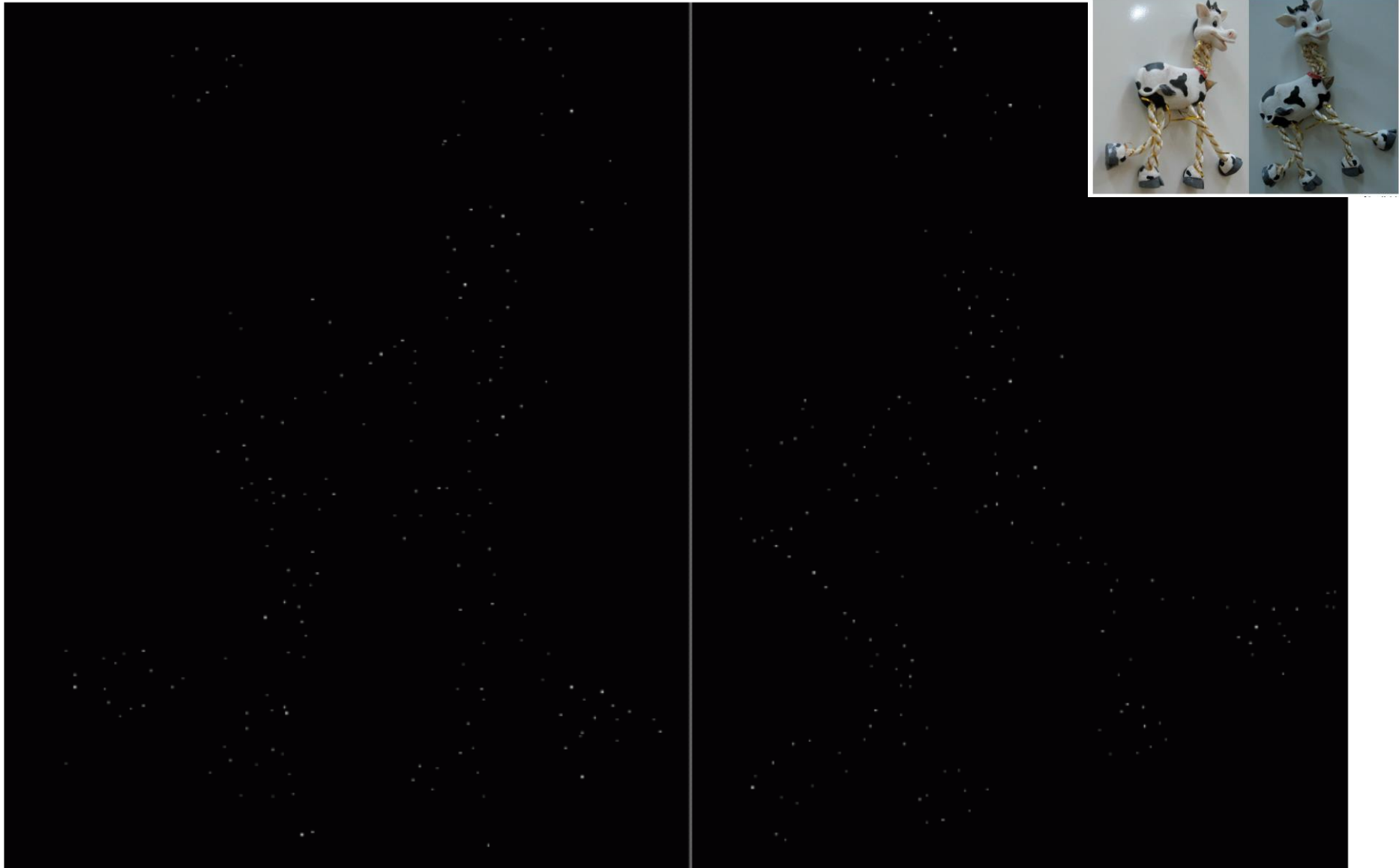
Non-maximum suppression



For every pixel p with a score greater than the corner threshold:
Check score of points within distance r
If p is the maximum within radius r , mark p as a corner

- Vary r until the desired number of corners are kept
 - Larger radius will suppress more pixels
 - Smaller radius will keep more

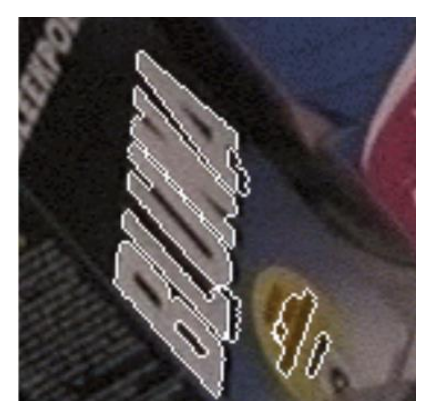
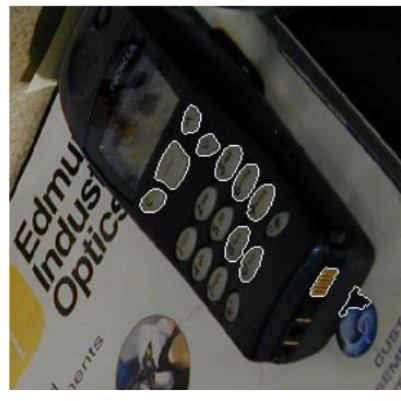
Non-maximum suppression



Harris corners in red



Maximally stable “extremal regions”



- Invariant to affine transformation of gray-values
- Both small and large structures are detected